

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH**



LÊ TRẦN MINH PHÚC

**TỐI ƯU CHẤT LƯỢNG - ĐA DẠNG CÓ ĐIỀU KIỆN NGŨ
NGHĨA TRONG KHÔNG GIAN TIỀM ẨN ĐỂ TẠO SINH
MÀN CHƠI THE LEGEND OF ZELDA**

**KHÓA LUẬN TỐT NGHIỆP
NGÀNH KHOA HỌC MÁY TÍNH**

TP. HỒ CHÍ MINH, 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH**



LÊ TRẦN MINH PHÚC

**TỐI ƯU CHẤT LƯỢNG - ĐA DẠNG CÓ ĐIỀU KIỆN NGŨ
NGHĨA TRONG KHÔNG GIAN TIỀM ẨN ĐỂ TẠO SINH
MÀN CHƠI THE LEGEND OF ZELDA**

Chuyên ngành: Khoa học máy tính
Mã số sinh viên: 2251010074 Lớp: DH22CS01C

**KHÓA LUẬN TỐT NGHIỆP
NGÀNH KHOA HỌC MÁY TÍNH**

Giảng viên hướng dẫn: **TS. Lê Đăng Giáp**

TP. HỒ CHÍ MINH, 2026

TRƯỜNG ĐẠI HỌC MỞ
THÀNH PHỐ HỒ CHÍ MINH
KHOA Đào Tạo Đặc Biệt

CỘNG HÒA XÃ HỘI CHỦ NGHĨA
VIỆT NAM
Độc lập - Tự do - Hạnh phúc

Thành phố Hồ Chí Minh, ngày tháng
..... năm 2026

GIẤY XÁC NHẬN

Tôi tên là: Lê Trần Minh Phúc

Ngày sinh: 18/04/2004

Nơi sinh: Tây Ninh

Chuyên ngành: Khoa học máy tính

Mã sinh viên: 2251010074

Tôi đồng ý cung cấp toàn văn thông tin khóa luận tốt nghiệp hợp lệ và bản quyền cho Thư viện Trường Đại học Mở Thành phố Hồ Chí Minh. Thư viện Trường Đại học Mở Thành phố Hồ Chí Minh được phép nộp toàn văn thông tin khóa luận vào hệ thống thông tin khoa học theo quy định hiện hành.

Ký tên

(Ghi rõ họ và tên)

Lê Trần Minh Phúc

Ý KIẾN CHO PHÉP BẢO VỆ KHÓA LUẬN TỐT NGHIỆP

Giảng viên hướng dẫn: TS. Lê Đăng Giáp

Sinh viên thực hiện: Lê Trần Minh Phúc

Lớp: DH22CS01C

Ngày sinh: 18/04/2004

Nơi sinh: Tây Ninh

Tên khóa luận: Tối ưu chất lượng - đa dạng có điều kiện ngữ nghĩa trong không gian tiềm ẩn để tạo sinh màn chơi The Legend of Zelda

Ý kiến của giảng viên hướng dẫn về việc cho phép sinh viên được bảo vệ khóa luận tốt nghiệp trước Hội đồng:

Thành phố Hồ Chí Minh, ngày tháng năm 2026

Giảng viên hướng dẫn

TS. Lê Đăng Giáp

LỜI CẢM ƠN

Em xin chân thành cảm ơn TS. Lê Đăng Giáp đã tận tình hướng dẫn, góp ý và đồng hành cùng em trong suốt quá trình thực hiện khóa luận.

Em xin gửi lời cảm ơn đến TS. Sanggyu Nam vì những trao đổi chuyên môn và góp ý quý báu, giúp em mở rộng góc nhìn khi triển khai đề tài. Những chỉ dẫn của thầy giúp em định hướng rõ hơn cách tiếp cận đề tài và hoàn thiện nội dung nghiên cứu.

Em cũng xin cảm ơn gia đình và bạn bè đã luôn động viên, hỗ trợ và tạo điều kiện để em yên tâm học tập, nghiên cứu và hoàn thành khóa luận này.

Do thời gian và kinh nghiệm còn hạn chế, khóa luận chắc chắn vẫn còn thiếu sót. Em rất mong nhận được những góp ý của người đọc và các bạn để em có thể hoàn thiện hơn trong những nghiên cứu tiếp theo.

Trân trọng,

Lê Trần Minh Phúc

Tóm tắt

Khóa luận này nghiên cứu bài toán tạo sinh dungeon cho *The Legend of Zelda* theo hướng kết hợp mô hình học máy với các ràng buộc ký hiệu nhằm bảo đảm tiến trình chơi và tính hợp lệ của cấu trúc. Thay vì sinh trực tiếp toàn bộ bản đồ, hệ thống được tổ chức theo hướng tách tầng: trước hết sinh cấu trúc đồ thị nhiệm vụ, sau đó sinh chi tiết từng phòng trong không gian tiềm ẩn rời rạc. Kiến trúc đề xuất gồm bảy khối chính: bộ xử lý dữ liệu, bộ sinh cấu trúc đồ thị nhiệm vụ bằng tiến hóa, VQ-VAE ngữ nghĩa, bộ mã hóa điều kiện hai luồng, mô hình diffusion trong latent space, mô-đun hướng dẫn logic, bộ sửa lỗi ký hiệu và lớp đánh giá quality-diversity.

Cách tổ chức này cho phép mô hình duy trì quan hệ khóa – chìa, kết nối phòng và khả năng chơi được, đồng thời vẫn giữ được độ đa dạng của bố cục phòng. Hệ thống được kiểm chứng bằng các thí nghiệm ablation, so sánh theo matched-budget và các kịch bản thay đổi quy mô đồ thị nhiệm vụ để đánh giá mức độ ổn định của từng thành phần. Kết quả thực nghiệm cho thấy kiến trúc lai giúp cân bằng tốt hơn giữa tính hợp lệ, khả năng giải được và độ đa dạng nội dung so với các cấu hình không tách rõ vai trò giữa tầng đồ thị nhiệm vụ và tầng sinh phòng.

Từ đó, khóa luận khẳng định tính khả thi của hướng tiếp cận graph-first kết hợp latent diffusion và sửa lỗi ký hiệu cho bài toán tạo sinh dungeon Zelda trong bối cảnh dữ liệu hạn chế.

Từ khóa: sinh nội dung thủ tục, dungeon Zelda, topology-first, VQ-VAE, latent diffusion, sửa lỗi ký hiệu, quality-diversity.

Abstract

This thesis investigates dungeon generation for *The Legend of Zelda* through a hybrid neural-symbolic framework that separates mission topology from room synthesis. Rather than generating an entire level in a single step, the proposed system first constructs a mission graph that encodes progression, locks, branches, and target structure, and then generates each room in a semantic latent space before applying symbolic post-processing to repair residual inconsistencies. The architecture combines seven components: data preprocessing, evolutionary topology search, a semantic VQ-VAE, a dual-stream condition encoder, latent diffusion, logic guidance, symbolic repair, and a quality-diversity evaluation layer.

This factorization is designed to preserve solvability and structural validity while still allowing diverse room layouts and controllable game progression. The thesis evaluates the system through fixed-seed ablations, matched-budget comparisons, and topology-scale tests in order to isolate the contribution of each block. The experimental protocol focuses on structural correctness, playability, robustness, and behavioral diversity rather than on visual fidelity alone.

Overall, the results indicate that the hybrid design provides a practical compromise between global control and local generative flexibility for Zelda-style dungeon generation. More broadly, the work shows that combining graph-first planning, latent generative modeling, and symbolic repair is a viable research direction for procedural content generation under limited data.

Keywords: procedural content generation, Zelda dungeon generation, topology-first design, VQ-VAE, latent diffusion, symbolic repair, quality-diversity.

Mục lục

Tóm tắt		
Abstract		
Danh sách hình		v
Danh sách bảng		vi
Danh sách từ viết tắt (List of Abbreviations)		vii
Danh mục thuật ngữ chuyên ngành		viii
1 Giới thiệu		1
1.1 Lý do chọn đề tài		1
1.2 Mục tiêu nghiên cứu		2
1.3 Đối tượng và phạm vi nghiên cứu		2
1.4 Phương pháp nghiên cứu		3
1.5 Câu hỏi nghiên cứu		4
1.6 Đóng góp của luận văn		4
1.7 Bố cục luận văn		5
2 Tổng quan nghiên cứu và cơ sở lý thuyết		6
2.1 Nền tảng lý thuyết		6
2.1.1 Sinh nội dung thủ tục trong trò chơi		6
2.1.2 Biểu diễn rời rạc và sinh trong không gian tiềm ẩn		7
2.1.3 Điều kiện hóa theo đồ thị nhiệm vụ		7
2.1.4 Khung khái niệm và chuẩn hóa thuật ngữ		8
2.1.5 Tiêu chí phân tích và đánh giá		8

2.2	Tổng quan công trình liên quan	9
2.2.1	Hướng đồ thị nhiệm vụ và PCG dựa trên tìm kiếm	9
2.2.2	Hướng sinh đồ thị học sâu và diffusion rời rạc	9
2.2.3	Hướng PCGML ở cấp độ phòng	10
2.2.4	Hướng lai nơ-ron – ký hiệu	11
2.3	Khoảng trống nghiên cứu	11
2.4	Kết luận chương	12
3	Phương pháp đề xuất	13
3.1	Mục tiêu và phạm vi chương	13
3.2	Kiến trúc tổng thể và hợp đồng dữ liệu	14
3.2.1	Phân biệt architecture graph và mission graph	14
3.2.2	Mô hình kiến trúc đồ thị (architecture graph) 7 khối	15
3.2.3	Ma trận hợp đồng dữ liệu theo khối	16
3.2.4	Dòng chảy đầu-cuối của quy trình	18
3.2.5	Biểu diễn dữ liệu và schema đầu vào	19
3.2.6	Tiền xử lý dữ liệu và chính sách chia tập	19
3.2.7	Ràng buộc triển khai và bất biến dữ liệu	20
3.3	Thiết kế chi tiết các khối trong pipeline	21
3.3.1	Nguyên tắc lựa chọn thiết kế giữa các phương án thay thế	21
3.3.2	Mô hình sinh mission graph (Khối I): cấu trúc và tiến trình	21
3.3.3	Khối mã hóa điều kiện graph-to-room (Khối III)	24
3.3.4	Khối sinh nơ-ron: VQ-VAE (Khối II) và latent diffusion (Khối IV)	25
3.3.5	Khối hướng dẫn logic và sửa lỗi ký hiệu (Khối V–VI)	25
3.4	Thuật toán huấn luyện và suy luận trong mã nguồn	27
3.4.1	Huấn luyện theo giai đoạn	27
3.4.2	Suy luận nhiều phòng và cơ chế fallback	29
3.4.3	Đánh giá sau sinh và chỉ số vận hành	34

3.4.4	Giả định vận hành, chế độ lỗi và biện pháp giảm thiểu	35
3.5	Cấu hình triển khai và khả năng tái lập	36
3.6	Ánh xạ nội dung phương pháp vào luồng hiện thực	37
3.7	Liên kết với thiết kế thực nghiệm Chương 4	37
3.8	Tóm tắt chương	38
4	Kết quả thực nghiệm và thảo luận	39
4.1	Mục tiêu kiểm chứng và câu hỏi thực nghiệm	39
4.2	Thiết kế thực nghiệm	40
4.2.1	Thiết lập tái lập và môi trường thực nghiệm	40
4.2.2	Dữ liệu và kịch bản đánh giá	40
4.2.3	Baseline và cấu hình so sánh	40
4.2.4	Chỉ số đánh giá và suy luận thống kê	40
4.2.5	Giao thức thí nghiệm	40
4.3	Kết quả định lượng	41
4.3.1	Kết quả tổng thể so với baseline	41
4.3.2	Kết quả ablation theo khối pipeline	41
4.3.3	Kết quả matched-budget ở tầng đồ thị nhiệm vụ	41
4.3.4	Kết quả OOD theo quy mô dungeon	41
4.3.5	Độ tin cậy vận hành và chi phí	41
4.4	Thảo luận	41
4.4.1	Giải thích cơ chế cải thiện	41
4.4.2	Đánh đổi chất lượng, đa dạng và chi phí	42
4.4.3	Mối đe dọa tính hiệu lực và giới hạn	42
4.5	Tóm tắt chương	42
5	Kết luận và hướng phát triển	43
	PHỤ LỤC	44

A	Khung ký hiệu và mô hình chi phí	44
A.1	Ký hiệu sử dụng trong phụ lục	44
A.2	Mô hình chi phí chuẩn hóa	44
B	Chứng minh độ phức tạp	44
B.1	Bổ đề phân lớp topo	44
B.2	Mệnh đề cho sinh cấu trúc đồ thị tiến hóa	45
B.3	Mệnh đề cho lập lịch nhiều phòng	45
B.4	Mệnh đề cho huấn luyện một epoch diffusion	46
B.5	Mệnh đề cho suy luận một phòng với teacher fallback	46
B.6	Mệnh đề cho đánh giá hậu kỳ sau ghép dungeon	46
C	Phép tính minh họa cho độ phức tạp thực toán	47
C.1	Nguyên tắc tính minh họa	47
C.2	Ví dụ 1: chi phí sửa một phòng	47
C.3	Ví dụ 2: chi phí lập lịch nhiều phòng	47
C.4	Ví dụ 3: ngân sách đánh giá ở tầng đồ thị nhiệm vụ	48
C.5	Ví dụ 4: kỳ vọng chi phí khi có fallback	48

Danh sách hình

3.1	Kiến trúc 7 khối của quy trình sinh topology-first neural-symbolic. Mũi tên nét đứt biểu diễn các nhánh hướng dẫn logic và đánh giá tùy cấu hình.	16
3.2	Luồng nội bộ của quy trình sinh phòng, bao gồm sửa lỗi ký hiệu và cơ chế teacher fallback.	29
3.3	Luồng lập lịch sinh phòng theo lớp phụ thuộc và bucket latent shape.	30

Danh sách bảng

3.1	Ký hiệu dùng xuyên suốt Chương 3	14
3.2	Ma trận hợp đồng dữ liệu và cơ chế an toàn theo từng khối	16
3.3	Các chế độ lỗi chính và cơ chế giảm thiểu trong đường chạy Chương 3	36

Danh sách từ viết tắt (List of Abbreviations)

Từ viết tắt	Ý nghĩa
AI	Artificial Intelligence
CFG	Classifier-Free Guidance
CMA-ME	Covariance Matrix Adaptation MAP-Elites
CVT	Centroidal Voronoi Tessellation
CVT-MAP-Elites	Centroidal Voronoi Tessellation MAP-Elites
DDIM	Denoising Diffusion Implicit Model
DDPM	Denoising Diffusion Probabilistic Model
GNN	Graph Neural Network
GPS	Graph Positioning System (Graph Transformer Recipe)
LCM	Latent Consistency Model
LDM	Latent Diffusion Model
MAP-Elites	Multi-dimensional Archive of Phenotypic Elites
PCG	Procedural Content Generation (Sinh nội dung thủ tục)
PCGML	Procedural Content Generation via Machine Learning
QD	Quality-Diversity
RQ	Research Question
VGLC	Video Game Level Corpus
VQ-VAE	Vector Quantized Variational Autoencoder
WFC	Wave Function Collapse

Danh mục thuật ngữ chuyên ngành

Thuật ngữ	Định nghĩa ngắn gọn
Topology-first generation	Chiến lược sinh nội dung theo thứ tự: xây dựng cấu trúc đồ thị nhiệm vụ (mission topology) trước, sau đó mới sinh chi tiết phòng.
Mission graph	Đồ thị mô tả các phòng/nút nhiệm vụ và quan hệ kết nối, điều kiện mở khóa trong dungeon.
Room layout	Bố cục tile bên trong một phòng, thể hiện tường, cửa, vật cản, thực thể và đường đi.
Latent space	Không gian biểu diễn nén dùng để mã hóa đặc trưng ngữ nghĩa của phòng trước khi sinh mẫu mới.
Discrete latent code	Mã biểu diễn rời rạc trong latent space, thường được học bởi VQ-VAE cho dữ liệu tile.
Conditional diffusion	Mô hình diffusion sinh mẫu dưới điều kiện đầu vào (ví dụ: ngữ cảnh phòng, ràng buộc đồ thị).
Graph conditioning	Cơ chế đưa thông tin từ mission graph vào mô hình sinh để giữ nhất quán tiến trình toàn cục.
Semantic constraint	Ràng buộc ngữ nghĩa gameplay, ví dụ tính hợp lệ khóa – chìa, boss room hoặc thứ tự mở cửa.
Logic guidance	Thành phần hướng dẫn quá trình sinh dựa trên luật và tín hiệu logic để giảm mẫu vi phạm.
Symbolic repair	Bước hậu xử lý dùng luật ký hiệu để sửa các lỗi cấu trúc hoặc tiến trình còn sót lại.
Solvability	Mức độ mà dungeon có thể được hoàn thành theo luật chơi và trạng thái vật phẩm hợp lệ.
Validity	Mức độ tuân thủ của dungeon đối với các ràng buộc hình học, kết nối và tiến trình nhiệm vụ.
Quality-Diversity (QD)	Họ phương pháp tối ưu đồng thời chất lượng nghiệm và độ đa dạng nghiệm trong không gian hành vi.
Ablation study	Thiết kế thực nghiệm tắt/bật từng thành phần để định lượng đóng góp của mỗi mô-đun.

Reproducibility

Khả năng tái lập kết quả thực nghiệm dưới cùng cấu hình, seed và dữ liệu đầu vào.

CHƯƠNG 1

Giới thiệu

Chương này mở đầu luận văn bằng cách trình bày bối cảnh, động cơ và mục tiêu nghiên cứu. Nội dung chương cũng xác định phạm vi, câu hỏi nghiên cứu và các đóng góp chính, làm cơ sở cho phần nền tảng lý thuyết và phương pháp ở các chương tiếp theo.

1.1 Lý do chọn đề tài

Trong phát triển trò chơi hiện đại, nhu cầu mở rộng nội dung với chi phí hợp lý khiến sinh nội dung thủ tục (Procedural Content Generation – PCG) trở thành một hướng nghiên cứu có ý nghĩa thực tiễn rõ rệt [1]. Tuy nhiên, với bài toán dungeon, giá trị của hệ sinh không chỉ nằm ở số lượng mẫu tạo ra mà còn ở mức độ bảo toàn tiến trình nhiệm vụ và khả năng chơi được trong điều kiện ràng buộc cụ thể [2, 3].

Đối với *The Legend of Zelda*, dungeon là một cấu trúc nhiệm vụ có logic nội tại, trong đó quan hệ giữa cửa, khóa, chìa và mục tiêu phải nhất quán từ đầu đến cuối. Nếu hình thái phòng được tạo ra tốt về bề mặt nhưng không tương thích với đồ thị nhiệm vụ, người chơi có thể rơi vào trạng thái bế tắc hoặc hoàn thành theo cách trái với thiết kế. Trong nhóm công trình cùng miền *Zelda*, các tiếp cận tiến hóa đã cho thấy tiềm năng tạo dungeon có thể chơi được và đã được kiểm chứng trên người chơi thật [4]; tuy nhiên, bài toán cân bằng giữa tính đúng đắn tiến trình, chất lượng bố cục và độ đa dạng vẫn còn mở. Trong bối cảnh đó, các tiếp cận thuần học máy thường mạnh ở khả năng học mẫu cục bộ nhưng khó đảm bảo ràng buộc toàn cục, còn các tiếp cận thuần ký hiệu lại kiểm soát tốt tiến trình nhưng hạn chế về độ phong phú hình thái [5, 6]. Khoảng trống này là lý do trực tiếp để luận văn lựa chọn kiến trúc lai nơ-ron – ký hiệu nhằm dung hòa năng

lực tạo mẫu và năng lực kiểm soát.

1.2 Mục tiêu nghiên cứu

Mục tiêu tổng quát của luận văn là xây dựng và kiểm chứng một hệ thống tạo sinh dungeon Zelda có khả năng cân bằng ba yêu cầu thường xung đột nhau: tính hợp lệ của gameplay, chất lượng cấu trúc phòng và độ đa dạng nội dung trong điều kiện dữ liệu huấn luyện giới hạn.

Để đạt mục tiêu này, nghiên cứu tổ chức quy trình tạo sinh theo hướng *topology-first*, nghĩa là xác lập cấu trúc nhiệm vụ trước khi sinh chi tiết phòng. Trên nền cấu trúc đồ thị nhiệm vụ đó, luận văn phát triển khối tạo phòng trong không gian tiềm ẩn rời rạc bằng VQ-VAE kết hợp diffusion có điều kiện để tăng độ ổn định của mẫu sinh và giữ được ngữ nghĩa tile [7, 8, 9]. Song song với thành phần sinh, hệ thống được tích hợp cơ chế hướng dẫn logic và sửa lỗi ký hiệu nhằm giảm sai lệch về đường đi, quan hệ khóa – chìa và thứ tự tiến trình. Toàn bộ kiến trúc được đánh giá bằng thực nghiệm đối chứng theo hướng ablation để làm rõ đóng góp của từng thành phần và mức đánh đổi giữa chất lượng với đa dạng. Bên cạnh đó, nghiên cứu đặt mục tiêu đánh giá matched-budget giữa các phương pháp sinh đồ thị nhiệm vụ và kiểm tra khả năng khái quát theo các chế độ in-distribution, OOD-small, OOD-large. Kỳ vọng thực nghiệm là nâng tỉ lệ dungeon hợp lệ, duy trì khả năng giải được ở mức cao và hạn chế hiện tượng lặp mẫu.

1.3 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của luận văn là bài toán tạo sinh dungeon Zelda từ đồ thị nhiệm vụ và bố cục phòng dạng lưới tile. Phạm vi nghiên cứu được giới hạn trong miền dữ liệu Zelda cùng các ràng buộc gameplay cốt lõi, bao gồm kết nối phòng, quan hệ khóa – chìa và điều kiện hoàn thành nhiệm vụ.

Về mặt kỹ thuật, luận văn tập trung vào chuỗi xử lý lai nơ-ron – ký hiệu gồm bốn lớp

chính: sinh cấu trúc đồ thị nhiệm vụ, sinh phòng có điều kiện ngữ nghĩa trong không gian tiềm ẩn, hậu kiểm tra logic và sửa lỗi ký hiệu. Về mặt đánh giá, nghiên cứu ưu tiên các chỉ số định lượng liên quan đến tính hợp lệ, khả năng giải được, chất lượng và đa dạng. Luận văn không triển khai khảo sát người dùng quy mô lớn; thay vào đó, nghiên cứu chuẩn hóa bộ dữ liệu đánh giá mù (blinded packet) để phục vụ bước đánh giá người dùng ở giai đoạn mở rộng.

1.4 Phương pháp nghiên cứu

Luận văn sử dụng phương pháp nghiên cứu hỗn hợp, kết hợp giữa tổng quan tài liệu, thiết kế hệ thống và thực nghiệm định lượng. Ở bước đầu, các hướng nghiên cứu then chốt trong PCG, PCGML, sinh đồ thị nhiệm vụ, diffusion trong không gian tiềm ẩn và hệ lai nơ-ron – ký hiệu được hệ thống hóa để xác định cơ sở lý thuyết và khoảng trống nghiên cứu [1, 10, 11].

Từ nền tảng đó, kiến trúc được thiết kế theo hướng mô-đun hóa, làm rõ vai trò của từng khối và giảm phụ thuộc chéo để thuận lợi cho kiểm chứng thành phần. Hệ thống sau thiết kế được hiện thực thành quy trình đầu-cuối trên mã nguồn luận văn với cấu hình có kiểm soát nhằm bảo đảm khả năng tái lập. Ở lớp đánh giá, nghiên cứu sử dụng giao thức fixed-seed, so sánh matched-budget và phân tích theo miền OOD để kiểm tra tính ổn định khi thay đổi quy mô đồ thị nhiệm vụ. Ở lớp suy luận thống kê, luận văn dùng paired bootstrap để ước lượng khoảng tin cậy 95% [12], kiểm định hoán vị dấu theo cặp để ước lượng p-value, và hiệu chỉnh đa kiểm định theo Benjamini-Hochberg FDR [13]. Cuối cùng, nghiên cứu phân tích kết quả trên các nhóm chỉ số hợp lệ, giải được, chất lượng và đa dạng để rút ra kết luận thực chứng.

1.5 Câu hỏi nghiên cứu

Từ mục tiêu và phạm vi nghiên cứu đã nêu, luận văn tập trung trả lời bốn vấn đề cốt lõi theo hướng định lượng. Trước hết, nghiên cứu kiểm định liệu chiến lược *topology-first* có cải thiện đồng thời tính hợp lệ gameplay và chất lượng tạo phòng so với cấu hình không tách tầng hay không. Tiếp theo, luận văn phân tích thành phần nào giữ vai trò chi phối trong pipeline đề xuất, bao gồm sinh cấu trúc đồ thị nhiệm vụ (topology evolution), VQ-VAE, diffusion có điều kiện theo đồ thị, cơ chế hướng dẫn logic và sửa lỗi ký hiệu. Một vấn đề quan trọng khác là mức độ đóng góp thực sự của hậu kiểm tra và sửa lỗi ký hiệu đối với độ tin cậy của dungeon trong bối cảnh dữ liệu nhỏ. Cuối cùng, nghiên cứu làm rõ liệu lợi thế của phương pháp đề xuất còn được duy trì khi so sánh matched-budget với các baseline sinh đồ thị nhiệm vụ và khi chuyển sang các miền OOD theo số lượng phòng hay không.

1.6 Đóng góp của luận văn

Đóng góp của luận văn thể hiện trên cả phương diện kiến trúc, hiện thực hệ thống và đánh giá thực nghiệm. Trên phương diện kiến trúc, nghiên cứu đề xuất mô hình lai nơ-ron – ký hiệu có phân tầng rõ giữa lớp nhiệm vụ và lớp tạo phòng, qua đó tăng khả năng kiểm soát tiến trình gameplay. Trên phương diện hiện thực, luận văn xây dựng khối tạo phòng trong không gian tiềm ẩn rời rạc có điều kiện ngữ nghĩa và tích hợp cơ chế hướng dẫn logic cùng sửa lỗi ký hiệu để nâng cao tỉ lệ dungeon hợp lệ, giải được. Trên phương diện thực nghiệm, nghiên cứu thiết kế giao thức gồm fixed-seed ablation có kiểm định thống kê, so sánh matched-budget giữa các phương pháp sinh đồ thị nhiệm vụ, đánh giá OOD scaling theo quy mô phòng, và chuẩn hóa bộ blinded packet cho bước đánh giá người dùng mở rộng.

1.7 Bố cục luận văn

Phần còn lại của khóa luận được tổ chức thành bốn chương. Chương 2 trình bày tổng quan nghiên cứu và cơ sở lý thuyết, đồng thời xác định khoảng trống mà đề tài hướng tới. Chương 3 mô tả chi tiết kiến trúc đề xuất, luồng dữ liệu và cách hiện thực các mô-đun trong hệ thống. Chương 4 trình bày thiết kế thực nghiệm, giao thức đánh giá, kết quả định lượng và phân thảo luận. Chương 5 tổng kết các đóng góp chính, nêu hạn chế và đề xuất hướng phát triển tiếp theo.

CHƯƠNG 2

Tổng quan nghiên cứu và cơ sở lý thuyết

Chương này hệ thống hóa các nền tảng lý thuyết và các hướng nghiên cứu liên quan trực tiếp đến bài toán tạo sinh dungeon Zelda. Cách trình bày đi từ các khái niệm nền tảng của PCG và mô hình sinh rời rạc, sang các hướng nghiên cứu về điều kiện hóa theo đồ thị, rồi tổng hợp các công trình liên quan và khoảng trống nghiên cứu làm cơ sở cho kiến trúc đề xuất ở Chương 3.

2.1 Nền tảng lý thuyết

Phần này tổng hợp các nền tảng cốt lõi làm cơ sở cho kiến trúc đề xuất, từ PCG truyền thống đến các mô hình sinh trong không gian tiềm ẩn. Cách trình bày đi từ bức tranh tổng quát đến các cơ chế cụ thể đang được hiện thực trong hệ thống.

2.1.1 Sinh nội dung thủ tục trong trò chơi

Trong nghiên cứu sinh nội dung thủ tục (PCG), ba hướng tiếp cận được dùng nhiều nhất là dựa trên luật, dựa trên tìm kiếm và dựa trên học máy tạo sinh [1, 10, 14]. Mỗi hướng có thể mạnh riêng: phương pháp dựa trên luật thuận lợi cho kiểm soát ràng buộc, phương pháp tìm kiếm phù hợp với tối ưu đa mục tiêu, còn phương pháp học máy mạnh ở khả năng học mẫu không gian phức tạp từ dữ liệu.

Ở nhánh học máy cho PCG, tổng quan PCGML của Summerville et al. cho thấy bài toán không chỉ nằm ở sinh mẫu mới mà còn ở khả năng sửa lỗi, phân tích nội dung và làm việc trong điều kiện dữ liệu nhỏ [15]. Nhận định này đặc biệt phù hợp với miền Zelda, nơi ràng buộc gameplay khiến chất lượng dữ liệu và cách biểu diễn có ảnh hưởng trực tiếp đến độ tin cậy của đầu ra.

Với dungeon Zelda, chất lượng không thể chỉ đánh giá bằng hình thức bản đồ. Một hệ tạo sinh phù hợp phải đồng thời bảo đảm logic tiến trình nhiệm vụ, khả năng hoàn thành theo luật chơi, và chất lượng bố cục ở mức tile [3]. Vì vậy, mục tiêu không chỉ là ”sinh được nhiều mẫu” mà là ”sinh được mẫu hợp lệ và có giá trị gameplay”.

2.1.2 Biểu diễn rời rạc và sinh trong không gian tiềm ẩn

Với dữ liệu phòng ở dạng lưới tile và từ vựng hữu hạn, biểu diễn rời rạc bằng VQ-VAE là lựa chọn phù hợp để học mã ngữ nghĩa [7]. Về nền tảng, cách xây dựng latent và tối ưu hóa mục tiêu tái tạo-khái quát hóa kế thừa trực tiếp từ khung học biểu diễn trong các giáo trình học máy hiện đại [16, 17]. Cách làm này nén thông tin cấu trúc vào không gian tiềm ẩn gọn hơn, từ đó giảm độ khó cho mô-đun sinh phía sau và tăng tính ổn định khi dữ liệu huấn luyện không lớn.

Trên biểu diễn latent rời rạc, diffusion đóng vai trò bộ sinh chính để tạo mẫu có chất lượng cục bộ tốt và đa dạng hình thái [8, 9, 11]. So với sinh trực tiếp trên lưới tile gốc, cách tiếp cận này thường thuận lợi hơn khi cần giữ latent gọn và gắn điều kiện ngữ nghĩa.

Bên cạnh diffusion trong không gian liên tục, hướng diffusion rời rạc cũng đáng chú ý vì làm việc trực tiếp trên token/categorical state. D3PM cho thấy lựa chọn ma trận chuyển trạng thái và cơ chế nhiễu rời rạc có ảnh hưởng rõ đến chất lượng sinh mẫu [18]. Trên các bài toán bố cục có ràng buộc, LayoutDM tiếp tục chỉ ra rằng diffusion rời rạc kết hợp cơ chế tiêm điều kiện ở pha suy luận có thể nâng đáng kể khả năng kiểm soát cấu trúc đầu ra [19].

2.1.3 Điều kiện hóa theo đồ thị nhiệm vụ

Trong bài toán dungeon, ngữ cảnh phải được xét đồng thời ở hai tầng: cục bộ (liên kết biên, quan hệ phòng kề) và toàn cục (vai trò của phòng trong đồ thị nhiệm vụ). Nếu chỉ dùng thông tin cục bộ, mô hình có thể tạo ra phòng ”đẹp” nhưng lệch với tiến trình nhiệm vụ tổng thể.

Vì vậy, cơ chế điều kiện hóa đa nguồn và attention từ đồ thị sang lưới là lựa chọn được nhiều công trình liên quan hậu thuẫn [20, 5, 21]. Hướng này có thể giúp giảm tình trạng đúng hình thức nhưng sai chức năng, đồng thời tăng tính nhất quán gameplay giữa các phòng trong cùng một dungeon.

2.1.4 Khung khái niệm và chuẩn hóa thuật ngữ

Bên cạnh nhóm bài báo gốc, Chương 2 còn tham chiếu các giáo trình và tài liệu nền tảng để chuẩn hóa khung khái niệm, thuật ngữ và cách diễn giải. Ở trục PCG, giáo trình của Shaker, Togelius và Nelson cung cấp hệ quy chiếu nhất quán cho các chiến lược rule-based, search-based và data-driven khi đánh giá hệ tạo sinh [14]. Ở trục học máy, các giáo trình kinh điển về học biểu diễn và mô hình xác suất giúp cố định cách hiểu về latent space, regularization và đánh đổi giữa khớp dữ liệu với khả năng khái quát hóa [16, 17]. Việc kết hợp hai lớp nguồn này giúp phần cơ sở lý thuyết vừa bám sát các công trình hiện đại, vừa giữ được nền tảng phương pháp luận ổn định cho các chương sau.

2.1.5 Tiêu chí phân tích và đánh giá

Đối với hệ lai nơ-ron – ký hiệu, phần lý thuyết không dừng ở việc mô tả mô hình sinh mà còn cần chỉ ra cách đọc kết quả. Các công trình liên quan thường xem xét đồng thời tính hợp lệ cấu trúc, khả năng chơi được, độ đa dạng, độ ổn định khi thay đổi ngân sách và mức độ bền vững trước nhiễu hoặc dịch chuyển miền dữ liệu. Với dungeon Zelda, các tiêu chí này đặc biệt quan trọng vì một mẫu có thể nhìn hợp lý ở mức cục bộ nhưng vẫn thất bại ở mức tiến trình nhiệm vụ hoặc tạo ra các nghẽn đường đi khó sửa. Cách nhìn theo nhiều trục này là nền tảng để luận văn lựa chọn các phép so sánh phù hợp ở chương thực nghiệm, thay vì chỉ dựa vào một chỉ số đơn lẻ.

2.2 Tổng quan công trình liên quan

Phần này hệ thống các hướng nghiên cứu gần với bài toán của luận văn theo ba trục chính: đồ thị nhiệm vụ (mission graph), PCGML cấp phòng và hệ lai nơ-ron – ký hiệu. Qua đó, luận văn chỉ ra điểm mạnh, giới hạn và mức độ phù hợp của từng hướng đối với dungeon Zelda.

2.2.1 Hướng đồ thị nhiệm vụ và PCG dựa trên tìm kiếm

Các nghiên cứu theo hướng đồ thị nhiệm vụ (mission graph), mission-space và graph grammar cho thấy ưu điểm rõ rệt của việc mô hình hóa tiến trình nhiệm vụ trước, rồi mới hiện thực hóa hình học phòng [2]. Cách tổ chức này giúp ràng buộc gameplay được đưa vào sớm, thay vì xử lý hậu kỳ khi lỗi đã xuất hiện trên toàn bản đồ.

Trong cùng mạch nghiên cứu, các phương pháp PCG dựa trên tìm kiếm, đặc biệt là nhóm quality-diversity như MAP-Elites và CVT-MAP-Elites, cho phép tối ưu đồng thời chất lượng và độ đa dạng [22, 23, 24]. Điều này đặc biệt phù hợp với bài toán sinh dungeon, nơi mục tiêu không chỉ là tìm một nghiệm tốt nhất mà là xây dựng một tập nghiệm phong phú nhưng vẫn chơi được. Ở miền Zelda cụ thể, kết quả của Pereira et al. cũng củng cố hướng đánh giá theo tiến trình khóa-cổng và chỉ số tuyến tính/độ dư đường đi thay vì chỉ kiểm tra khả năng giải nhị phân [4].

2.2.2 Hướng sinh đồ thị học sâu và diffusion rời rạc

Ngoài các tiếp cận grammar và evolutionary search, hướng sinh đồ thị học sâu gần đây tập trung vào mô hình hóa trực tiếp phân bố trên đồ thị có nhãn nút/cạnh. Graphormer cho thấy các thiên kiến cấu trúc như khoảng cách đường đi ngắn nhất, centrality và edge-aware bias có tác động rõ rệt đến chất lượng biểu diễn đồ thị [25]. Trên nhánh tạo sinh, DiGress mở rộng diffusion sang không gian đồ thị rời rạc và được xem như một baseline tham chiếu quan trọng cho bài toán sinh đồ thị có thuộc tính phân loại [26].

Ở mức phương pháp tổng quát hơn, D3PM cung cấp nền tảng lý thuyết quan trọng cho diffusion trên không gian rời rạc và giúp lý giải vì sao các thiết kế chuyển trạng thái khác nhau có thể tạo ra chất lượng sinh mẫu khác nhau ngay cả khi cùng khung huấn luyện [18]. Điều này có giá trị tham chiếu trực tiếp khi đánh giá các biến thể sinh cấu trúc đồ thị nhiệm vụ bằng mô hình học.

Đối với luận văn này, nhóm công trình trên có vai trò chuẩn đối sánh phương pháp: chúng giúp định vị rõ điểm mạnh/yếu giữa “sinh cấu trúc đồ thị nhiệm vụ bằng grammar + search” và “sinh cấu trúc đồ thị nhiệm vụ bằng learned graph generator” trong cùng một ngân sách thực nghiệm. Vì vậy, Chương 4 sẽ sử dụng chúng như mốc tham chiếu khi thảo luận mức cạnh tranh của Block I theo matched-budget.

2.2.3 Hướng PCGML ở cấp độ phòng

Các công trình PCGML gần đây, trong đó có các biến thể diffusion cho dữ liệu hạn chế, đã báo cáo kết quả hứa hẹn ở bài toán sinh phòng cục bộ [10, 6]. Nhìn chung, các mô hình này có xu hướng học được mẫu hình không gian và tạo ra bố cục đa dạng.

Tổng quan PCGML cũng chỉ ra các vấn đề kéo dài như khan hiếm dữ liệu, khó đồng bộ nhiều lớp ngữ nghĩa và khó chuyển giao phong cách giữa miền dữ liệu khác nhau [15]. Các nghiên cứu gần đây ở miền bố cục có điều kiện như HouseDiffusion và LayoutDM cho thấy tiềm năng của mô hình sinh có kiểm soát ràng buộc, nhưng cũng nhấn mạnh rằng chất lượng hình học chưa đồng nghĩa với tính đúng tiến trình gameplay [6, 19].

Tuy vậy, nhiều thiết lập ở cấp độ phòng vẫn chưa xử lý đầy đủ quan hệ tiến trình giữa nhiều phòng trong một dungeon hoàn chỉnh. Nói cách khác, chất lượng cục bộ chưa tự động kéo theo tính đúng đắn toàn cục. Hạn chế này cho thấy cần một lớp điều phối cấu trúc ở mức cao hơn, thay vì chỉ tăng năng lực bộ sinh phòng.

2.2.4 Hướng lai nơ-ron – ký hiệu

Trong bối cảnh PCG, hướng lai nơ-ron – ký hiệu được quan tâm vì kết hợp được hai năng lực bổ sung nhau: khả năng học mẫu linh hoạt của mô hình nơ-ron và khả năng kiểm soát ràng buộc của thành phần ký hiệu [27, 2]. Với các bài toán có ràng buộc gameplay mạnh như Zelda, hướng này đặc biệt phù hợp.

Trong thực hành, hậu kiểm tra và sửa lỗi ký hiệu không chỉ là bước xử lý tình huống mà còn là một cơ chế quan trọng để nâng độ tin cậy của toàn quy trình; đặc biệt khi kết hợp với các bộ ràng buộc và thủ tục sửa cục bộ như WFC [28, 27].

2.3 Khoảng trống nghiên cứu

Từ các công trình đã khảo sát, có thể rút ra bốn khoảng trống chính. Thứ nhất, nhiều phương pháp hiện có chỉ tối ưu tốt một phía: hoặc kiểm soát tiến trình nhưng hạn chế độ giàu hình thái, hoặc sinh được phòng đa dạng nhưng chưa bảo toàn đầy đủ logic toàn cục của dungeon. Thứ hai, các công trình ở cấp độ phòng thường chưa giải quyết trọn vẹn quan hệ phụ thuộc giữa nhiều phòng trong cùng một mission graph, nên chất lượng cục bộ chưa tự động chuyển thành tính đúng đắn toàn cục. Thứ ba, bằng chứng so sánh giữa các hệ lai còn thiếu nhất quán vì khác nhau về ngân sách tính toán, phạm vi dữ liệu và cách đo lường. Thứ tư, nhiều pipeline chưa nêu rõ vai trò của từng lớp từ đồ thị nhiệm vụ, mô hình sinh phòng đến lớp hậu kiểm tra, khiến việc tái lập và diễn giải kết quả trở nên khó khăn.

Những khoảng trống này cho thấy cần một kiến trúc topology-first, trong đó đồ thị nhiệm vụ giữ vai trò điều phối toàn cục, mô hình sinh trong không gian tiềm ẩn xử lý chi tiết cục bộ, và lớp sửa lỗi ký hiệu bảo toàn tính chơi được của đầu ra. Theo hướng đó, luận văn không thay thế hoàn toàn các hướng search-based hay PCGML hiện có, mà kết hợp những ưu điểm của chúng trong một pipeline phân tầng có thể kiểm chứng, nhằm ưu tiên tính nhất quán của gameplay trước khi tối ưu độ phong phú hình thái.

2.4 Kết luận chương

Chương 2 đã trình bày nền tảng lý thuyết, tổng quan các hướng nghiên cứu liên quan và xác định khoảng trống mà luận văn hướng tới giải quyết. Trên cơ sở đó, Chương 3 sẽ trình bày chi tiết kiến trúc đề xuất, luồng dữ liệu giữa các khối chức năng và cách hiện thực trong mã nguồn.

CHƯƠNG 3

Phương pháp đề xuất

Chương này trình bày phương pháp đề xuất theo đúng hiện thực trong mã nguồn, nhằm mô tả nhất quán từ quyết định thiết kế đến cơ chế vận hành của hệ thống. Trọng tâm của chương là đặc tả kiến trúc, các đại lượng được tối ưu, luồng tín hiệu điều kiện giữa các khối và các ràng buộc an toàn giúp duy trì chất lượng đầu ra trong bối cảnh dữ liệu huấn luyện hạn chế.

3.1 Mục tiêu và phạm vi chương

Phương pháp được xây dựng để xử lý đồng thời hai mục tiêu: bảo toàn tiến trình gameplay ở mức toàn dungeon và duy trì chất lượng hình học ở mức từng phòng. Ở góc độ triển khai, hệ thống tuân theo quy trình *topology-first*: đồ thị nhiệm vụ được xác lập trước, sau đó mới sinh phòng có điều kiện theo đồ thị, rồi áp lớp hậu kiểm tra ký hiệu để khôi phục và duy trì ràng buộc gameplay. Cách tách tầng này phù hợp với hướng mission-space và nhóm phương pháp lai neural-symbolic trong PCG [2, 27].

Phạm vi chương tập trung vào các nhánh nằm trên đường chạy chính của huấn luyện và suy luận, gồm: sinh cấu trúc đồ thị nhiệm vụ bằng tiên hóa, mã hóa điều kiện hai luồng, khuếch tán trong không gian tiềm ẩn, cơ chế hướng dẫn logic và sửa lỗi ký hiệu. Các biến thể thử nghiệm ngoài đường chạy chính không được đưa vào đặc tả nhằm bảo toàn tính tái lập và tính kiểm chứng của phương pháp.

Để mạch trình bày rõ ràng, Chương 3 được tổ chức theo bốn lớp: (i) kiến trúc tổng thể và hợp đồng dữ liệu, (ii) thiết kế chi tiết từng khối, (iii) thuật toán vận hành huấn luyện/suy luận, và (iv) cấu hình triển khai – tái lập. Cấu trúc này giúp tách bạch giữa phần “vì sao thiết kế như vậy” và phần “cách hệ thống thực thi trong mã nguồn”.

Bảng ký hiệu cốt lõi cho Chương 3. Để giảm tải chuyển ngữ cảnh giữa các mục, Bảng 3.1 gom các ký hiệu được dùng lặp lại trong phần phương pháp.

Bảng 3.1: Ký hiệu dùng xuyên suốt Chương 3	
Ký hiệu	Ý nghĩa
$\mathcal{A}$	Architecture graph ở mức hệ thống (khối xử lý và luồng dữ liệu).
G	Mission graph ở mức nội dung dungeon (tiền trình nhiệm vụ, lock-key, nhánh, mục tiêu).
x_i	Lưới tile của phòng i trong không gian rời rạc 16×11 .
z_i	Biểu diễn latent của phòng i trong không gian liên tục do VQ-VAE mã hóa.
c_i	Vector điều kiện graph-to-room của phòng i sau khi hợp nhất nhánh cục bộ và toàn cục.
D	Dungeon đầu ra sau bước ghép phòng và hậu kiểm tra ký hiệu.
(f, v, ϕ)	Bộ chuẩn đoán cá thể topology gồm fitness, mức vi phạm ràng buộc và cờ khả thi.
$\mathcal{L}_{\text{diff}}, \mathcal{L}_{\text{logic}}$	Thành phần mất mát chính trong huấn luyện diffusion có điều kiện và hướng dẫn logic.
$\mathbb{I}_{\text{fallback}}$	Biến chỉ thị kích hoạt teacher fallback trong suy luận phòng.

3.2 Kiến trúc tổng thể và hợp đồng dữ liệu

3.2.1 Phân biệt architecture graph và mission graph

Để tránh nhập nhằng thuật ngữ, chương này dùng song song hai lớp đồ thị có vai trò khác nhau:

$$\mathcal{A} = (V_{\mathcal{A}}, E_{\mathcal{A}}), \quad G = (V_G, E_G). \quad (3.1)$$

Trong đó $\mathcal{A}$ là *architecture graph* ở mức hệ thống (nút là các khối xử lý, cạnh là luồng dữ liệu/chỉ đạo), còn G là *mission graph* ở mức nội dung dungeon (nút là phòng/trạng thái nhiệm vụ, cạnh là quan hệ tiến trình gameplay) [2]. Khối I có thể sinh mới G bằng

tiến hóa; các khối còn lại nhận G để điều kiện hóa sinh phòng, sửa lỗi, và đánh giá chất lượng.

Quy ước thuật ngữ cho Chương 3. Trong luận văn này, thuật ngữ “topology” được dùng theo nghĩa “cấu trúc đồ thị nhiệm vụ” trong mission-space PCG: bao gồm cấu trúc liên thông và các quan hệ tiến trình có ràng buộc (lock–key, item-gate, switch-state, boss gate) [2, 4, 1]. Khi chỉ phân tích hình thái đồ thị (số nút/cạnh, hệ số phân nhánh, chu trình, phân lớp topo), văn bản dùng cụm “sinh cấu trúc đồ thị” (topology generation) như tên rút gọn [29]. Khi phân tích thứ tự mở tiến trình và điều kiện vượt ải, văn bản dùng “ngữ nghĩa tiến trình” (progression semantics) hoặc “ràng buộc tiến trình nhiệm vụ” (mission progression constraints) để chỉ rõ đây là lớp ngữ nghĩa gameplay vượt ra ngoài nghĩa topology hẹp của lý thuyết đồ thị [2, 4]. Vì vậy, cách gọi đầy đủ cho Khối I trong ngữ cảnh phương pháp là “sinh cấu trúc và ngữ nghĩa tiến trình của đồ thị nhiệm vụ”, còn “topology generation” chỉ dùng ở các đoạn mô tả thuần hình thái cấu trúc.

3.2.2 Mô hình kiến trúc đồ thị (architecture graph) 7 khối

Để đặc tả kiến trúc ở mức cấu trúc, ta mô hình hóa pipeline bằng một đồ thị có hướng:

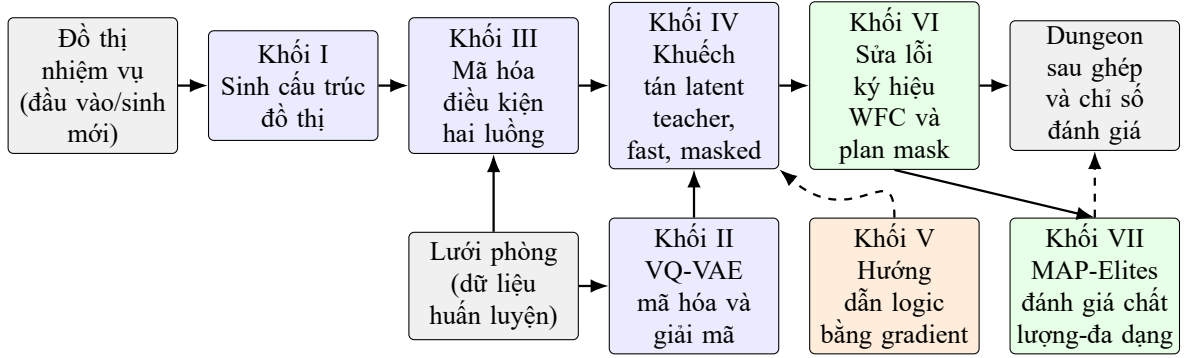
$$\mathcal{A} = (V_{\mathcal{A}}, E_{\mathcal{A}}), \quad V_{\mathcal{A}} = \{B_1, \dots, B_7\}, \quad (3.2)$$

trong đó B_1 đến B_7 lần lượt tương ứng các khối I đến VII ở Hình 3.1. Tập cạnh được tách thành hai phần:

$$E_{\mathcal{A}} = E_{\text{core}} \cup E_{\text{opt}}, \quad (3.3)$$

với E_{core} là các luồng bắt buộc trên đường chạy chính (cấu trúc đồ thị nhiệm vụ $\rightarrow$ conditioning $\rightarrow$ diffusion $\rightarrow$ repair), còn E_{opt} là các luồng tùy chọn dùng cho hướng dẫn logic hoặc đánh giá quality-diversity.

Trong phân rã chức năng, Khối I chịu trách nhiệm prior topology của G ; Khối II chuẩn hóa không gian latent; Khối III hợp nhất điều kiện cục bộ-toàn cục theo graph; Khối IV



Hình 3.1: Kiến trúc 7 khối của quy trình sinh topology-first neural-symbolic. Mũi tên nét đứt biểu diễn các nhánh hướng dẫn logic và đánh giá tùy cấu hình.

thực hiện lấy mẫu khuếch tán; Khối V bổ sung ràng buộc logic khả vi; Khối VI phục hồi ràng buộc ký hiệu ở tầng tile; và Khối VII đánh giá quality-diversity ở tầng dungeon. Cách tách khối này tạo ra giao diện rõ ràng cho ablation theo thành phần ở Chương 4.

3.2.3 Ma trận hợp đồng dữ liệu theo khối

Để bảo đảm truy vết được trách nhiệm của từng khối trong đường chạy chính, Bảng 3.2 tóm tắt hợp đồng vào-ra, cơ chế chặn lỗi và chỉ số giám sát tương ứng.

Bảng 3.2: Ma trận hợp đồng dữ liệu và cơ chế an toàn theo từng khối

Khối	Hợp đồng vào-ra cốt lõi	Cơ chế chặn lỗi/chuyển trạng thái	Mục tiêu hoặc chỉ số chính
B_1	Vào: luật, seed, profile ràng buộc. Ra: mission graph khả thi G và descriptor topology/progression.	Chọn nghiệm khả thi-ưu tiên; hậu kiểm tra liên thông, start-goal, virtual node.	Fitness đa mục tiêu, mức vi phạm v , cờ khả thi ϕ .

Khối	Hợp đồng vào-ra cốt lõi	Cơ chế chặn lỗi/chuyển trạng thái	Mục tiêu hoặc chỉ số chính
B_2	Vào: lưới phòng rời rạc. Ra: latent z và giải mã tile.	Kiểm tra shape chuẩn, bỏ batch non-finite.	$\mathcal{L}_{\text{recon}}$, $\mathcal{L}_{\text{VQ}}$, tỷ lệ tile hiếm.
B_3	Vào: $(X, E_{idx}, E_f, \text{TPE})$ cùng latent lân cận. Ra: điều kiện phòng c_i .	Kiểm tra schema tensor và fail-fast khi sai kích thước.	Độ ổn định điều kện hóa theo room layer, lỗi schema bằng 0.
B_4	Vào: z_t , timestep t , điều kện c . Ra: latent mẫu và dự đoán nhiễu/vận tốc.	Kẹp CFG theo miền distillation, Min-SNR weighting, clipping gradient.	$\mathcal{L}_{\text{diff}}$, tốc độ mẫu sinh, tỷ lệ fallback.
B_5	Vào: latent trung gian và plan trace. Ra: gradient logic và $\mathcal{L}_{\text{logic}}$.	Warmup theo epoch; tắt logic khi thành phần không khả dụng.	Grid reach, trace consistency, anchor consistency.
B_6	Vào: lưới nơ-ron, plan mask, marker đồ thị. Ra: lưới phòng đã sửa và hợp lệ biên-cửa.	Enforce boundary shell, WFC-guided repair, giới hạn số vòng sửa.	Repair rate, số lỗi cửa/đảo, hợp lệ kích thước phòng.

Khối	Hợp đồng vào-ra cốt lõi	Cơ chế chặn lỗi/chuyển trạng thái	Mục tiêu hoặc chỉ số chính
B_7	Vào: dungeon đã ghép và tín hiệu hợp lệ. Ra: cập nhật archive QD và metrics hậu sinh.	Chỉ thêm vào archive khi dungeon giải được; suy giảm mềm khi thành phần tắt.	Coverage, QD-score, linear- ity/leniency/progression complexity.

3.2.4 Dòng chảy đầu-cuối của quy trình

Gọi G là đồ thị nhiệm vụ, N là số phòng vật lý sau bước lọc nút ảo và D là dungeon đầu ra. Hệ thống thực hiện ánh xạ:

$$D = \mathcal{S}\left(\{\mathcal{R}(\mathcal{N}(G_i, G_i))\}_{i=1}^N\right), \quad (3.4)$$

trong đó $\mathcal{N}$ là khối sinh phòng nơ-ron có điều kiện, $\mathcal{R}$ là khối sửa lỗi ký hiệu theo ràng buộc cục bộ, và $\mathcal{S}$ là khối ghép phòng để tạo bản đồ toàn cục.

Trong thực thi, quy trình gồm ba lớp xử lý nối tiếp. Ở lớp đồ thị, hệ thống hoặc nhận đồ thị nhiệm vụ từ đầu vào hoặc tự sinh cấu trúc đồ thị nhiệm vụ bằng evolutionary search, rồi chuẩn hóa thứ tự nút để cố định ánh xạ giữa định danh phòng và chỉ số nút. Ở lớp phòng, các phòng được sinh theo lớp topo của đồ thị có hướng [29], sau đó nhóm theo latent shape để batch hóa mà không phá phụ thuộc predecessor. Ở lớp hậu xử lý, đầu ra neural được kiểm tra và sửa cục bộ theo plan mask trước khi ghép dungeon, nhờ đó tính nhất quán tiến trình được bảo toàn khi chuyển từ mức phòng sang mức toàn bản đồ.

3.2.5 Biểu diễn dữ liệu và schema đầu vào

Đồ thị nhiệm vụ được biểu diễn dưới hai dạng: vô hướng để duy trì liên thông toàn cục và có hướng để mô tả phụ thuộc tiến trình. Từ đồ thị này, hệ thống trích xuất tensor đặc trưng nút 14 chiều, đặc trưng cạnh 16 chiều và topological positional encoding 8 chiều. Không gian phòng sử dụng lưới rời rạc 16×11 với 44 lớp tile; sau đó Khối II ánh xạ sang latent chuẩn $64 \times 4 \times 3$ thông qua VQ-VAE [7]. Thiết kế ở mức latent giúp diffusion học phân bố ổn định hơn trong thiết lập dữ liệu phòng quy mô nhỏ, đồng thời vẫn duy trì khả năng can thiệp ký hiệu ở tầng tile [11]. Chi tiết cơ chế hợp nhất điều kiện cục bộ-toàn cục của Khối III được trình bày riêng trong Mục 3.3 để tách bạch lớp hợp đồng dữ liệu và lớp thiết kế thuật toán của từng khối.

3.2.6 Tiền xử lý dữ liệu và chính sách chia tập

Về tiền xử lý, pipeline chuẩn hóa dữ liệu về cùng một hợp đồng hình học-ngữ nghĩa trước khi huấn luyện: phòng được đưa về kích thước chuẩn 16×11 , tile id được ánh xạ vào từ vựng 44 lớp, và các kênh điều kiện theo biên/hàng xóm được dựng đồng nhất với schema ở Mục 3.2.5. Cùng lúc, dữ liệu đồ thị được chuẩn hóa thứ tự nút để tránh lệch chỉ số giữa pha huấn luyện và pha suy luận.

Để hạn chế rò rỉ thông tin giữa các tập, giao thức chia tập tuân thủ tính rời nhau theo nguồn dữ liệu:

$$\mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{val}} = \emptyset, \quad \mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{test}} = \emptyset, \quad \mathcal{D}_{\text{val}} \cap \mathcal{D}_{\text{test}} = \emptyset. \quad (3.5)$$

Ở mức phòng, các mẫu cùng xuất xứ đồ thị nhiệm vụ được ràng buộc vào cùng split:

$$\forall r_a, r_b : \text{src}(r_a) = \text{src}(r_b) \Rightarrow \text{split}(r_a) = \text{split}(r_b). \quad (3.6)$$

Nguyên tắc này giúp tránh trường hợp mô hình thấy một phần của cùng mission graph ở train và phần còn lại ở test, từ đó làm sai lệch kết luận về khả năng khái quát.

3.2.7 Ràng buộc triển khai và bất biến dữ liệu

Trong hiện thực hệ thống, chất lượng đầu ra không chỉ được kiểm soát bởi hàm mục tiêu mà còn bởi tập bất biến dữ liệu được kiểm tra xuyên suốt ở các điểm vào/ra của quy trình. Ở mức hình học phòng, trạng thái hợp lệ được duy trì bởi:

$$x_i \in \{0, \dots, 43\}^{16 \times 11}, \quad z_i \in \mathbb{R}^{64 \times 4 \times 3}. \quad (3.7)$$

Ở mức ràng buộc biên, vector điều kiện phòng được cố định 8 chiều theo thủ tục dựng ràng buộc biên:

$$b_i = [h_N, r_N, h_S, r_S, h_E, r_E, h_W, r_W] \in \{0, 1\}^8, \quad r_d \leq h_d \ (d \in \{N, S, E, W\}). \quad (3.8)$$

Gọi $\delta_d(\hat{x}_i) \in \{0, 1\}$ là biến chỉ thị “có cửa ở cạnh d ” trên phòng sinh ra $\hat{x}_i$, khi đó hợp đồng ràng buộc theo cạnh được viết:

$$r_d \leq \delta_d(\hat{x}_i) \leq h_d, \quad d \in \{N, S, E, W\}. \quad (3.9)$$

Biểu thức trên phản ánh đúng hành vi triển khai: cạnh không có hàng xóm bị ép thành tường biên; cạnh có cửa bắt buộc được khôi phục thông qua lớp sửa biên và lớp sửa ký hiệu trước khi trả kết quả phòng.

Ở mức graph-to-room conditioning, các tensor đầu vào được duy trì đúng schema:

$$X \in \mathbb{R}^{N \times 14}, \ E_{\text{idx}} \in \mathbb{N}^{2 \times M}, \ E_f \in \mathbb{R}^{M \times 16}, \ \text{TPE} \in \mathbb{R}^{N \times 8}, \quad (3.10)$$

và khi batch hóa cho diffusion phải thỏa:

$$\mathbf{B}_{\text{cond}} \in \mathbb{R}^{B \times 8}, \quad \mathbf{T}_{\text{topo}} \in \mathbb{R}^{B \times C \times H \times W}. \quad (3.11)$$

Các vi phạm shape tại các điểm này được phát hiện sớm bằng kiểm tra kiểu-kích thước và cơ chế báo lỗi sớm, thay vì để lỗi lan đến bước lấy mẫu; nhờ đó giảm đáng kể trường

hợp “sinh được nhưng sai hợp đồng dữ liệu”.

3.3 Thiết kế chi tiết các khối trong pipeline

3.3.1 Nguyên tắc lựa chọn thiết kế giữa các phương án thay thế

Để làm rõ phần “vì sao chọn thiết kế này”, luận văn dùng năm nguyên tắc xuyên suốt khi quyết định kiến trúc:

- (i) **Tách topology và hình học phòng:** ưu tiên kiểm soát tiến trình gameplay ở cấp đồ thị trước khi tối ưu chi tiết hình học ở cấp phòng.
- (ii) **Rời rạc hóa latent trước khi khuếch tán:** chọn VQ-VAE để nén ngữ nghĩa tile vào codebook ổn định, giảm độ khó cho khối diffusion trong thiết lập dữ liệu nhỏ.
- (iii) **Điều kiện hóa hai luồng cục bộ-toàn cục:** giữ đồng thời tín hiệu lân cận trực tiếp và vai trò phòng ở cấp dungeon để giảm xung đột biên-cửa.
- (iv) **Ổn định hóa huấn luyện hơn là tối ưu ngắn hạn:** dùng Min-SNR, warmup logic và kẹp CFG theo miền distillation để tránh nhiễu gradient ở giai đoạn đầu.
- (v) **An toàn suy luận theo lớp phòng thủ:** cho phép sampler nhanh hoạt động, nhưng luôn đặt hậu kiểm tra ký hiệu và teacher fallback làm tầng bảo hiểm cuối.

Năm nguyên tắc này cũng là cơ sở để thiết kế ablation ở Chương 4: mỗi nguyên tắc tương ứng một hoặc nhiều cờ bật/tắt độc lập, nhờ đó có thể ước lượng đóng góp biên của từng lớp quyết định kiến trúc.

3.3.2 Mô hình sinh mission graph (Khối I): cấu trúc và tiến trình

Khối I sử dụng bộ sinh mission graph tiến hóa dựa trên grammar, trong đó genotype là chuỗi luật và phenotype là đồ thị nhiệm vụ. Ở lớp mô tả hình thái (node/edge count, branching, cycles), mục này dùng cụm “sinh cấu trúc đồ thị” (topology generation); ở lớp ràng buộc khóa-công-vật phẩm-thứ tự vượt ải, mục này dùng “ngữ nghĩa tiến trình”

(progression semantics) hoặc “ràng buộc tiến trình nhiệm vụ” (mission progression constraints) [2, 4, 29]. Trong đối sánh với hướng học thuần đồ thị, DiGress là baseline quan trọng cho sinh đồ thị rời rạc có nhãn nút/cạnh [26]. Hàm thích nghi kết hợp độ khớp với đường cong mục tiêu (tension/difficulty), tính khả thi ràng buộc và các descriptor cấu trúc để tránh hội tụ sớm vào một kiểu đồ thị duy nhất. Trong cấu hình vận hành, không gian luật được mở rộng cùng cơ chế phối trộn chuyển trạng thái để cân bằng giữa prior kiểu Zelda và yêu cầu đa dạng cấu trúc, phù hợp với tinh thần quality-diversity của MAP-Elites/CVT [22, 23, 24].

Cơ chế chọn nghiệm và áp lực ràng buộc

Để làm tường minh lớp sinh cấu trúc đồ thị nhiệm vụ, ta mô hình hóa mỗi cá thể bằng bộ ba: chuỗi luật (genome), đồ thị sinh ra (phenotype), và bộ chuẩn đoán khả thi. Cụ thể:

$$g = (r_1, \dots, r_L), \quad \mathcal{G} = \text{EXECUTE}(g), \quad (f, v, \phi) = \text{EVALUATEGRAPH}(\mathcal{G}), \quad (3.12)$$

trong đó L là độ dài genome, $f \in [0, 1]$ là fitness, $v \geq 0$ là tổng mức vi phạm ràng buộc, và $\phi = \mathbf{1}[v \leq 10^{-9}]$ là cờ khả thi. Hàm EVALUATEGRAPH kết hợp độ khớp tension curve, descriptor cấu trúc và các hạng vi phạm (độ dài đường chính, dải số nút/cạnh, realism, directionality, rejection ratio, Pareto loops/branching), đúng với quy trình đánh giá đồ thị ở tầng hiện thực.

Trong bước chọn nghiệm, bộ sinh dùng thứ tự khả thi-ưu tiên kiểu Deb [30]:

$$\kappa(i) = (\mathbf{1}[\neg\phi_i], (1 - \phi_i) v_i, -f_i, \varepsilon_i), \quad (3.13)$$

với ε_i là lỗi realism topology. Cá thể có khóa từ điển nhỏ hơn sẽ được ưu tiên trong tournament và survivor selection, nên quần thể khả thi luôn được xếp trước quần thể không khả thi.

Thuật toán 1 là mô hình sinh cấu trúc đồ thị nhiệm vụ tường minh của Chương 3: vừa tối ưu mục tiêu chất lượng, vừa đảm bảo cơ chế áp lực ràng buộc xuyên suốt từ lúc áp

Thuật toán 1 Sinh cấu trúc đồ thị nhiệm vụ bằng GA/CVT-emitter với chọn nghiệm khả thi-ưu tiên

Đầu vào: Mục tiêu đường cong τ , cấu hình (P, G, L) , chiến lược $s \in \{GA, CVT\text{-emitter}\}$

Đầu ra: Đồ thị nhiệm vụ $\mathcal{G}^*$

```

1: Nếu  $s = CVT\text{-emitter}$  thì
2:    $B \leftarrow P \cdot G$  ▷ ngân sách đánh giá
3:   Khởi tạo CVT archive
4:   Đối với mỗi  $e = 1$  đến  $B$  thực hiện
5:     Phát genome từ emitter (init random hoặc archive-guided)
6:      $\mathcal{G} \leftarrow EXECUTE(g)$ ;  $(f, v, \phi) \leftarrow EVALUATEGRAPH(\mathcal{G})$ 
7:     Cập nhật archive và best theo khóa  $\kappa$ 
8:   end Đối với mỗi
9: Ngược lại
10:  Khởi tạo quần thể ban đầu kích thước  $P$ 
11:  Đối với mỗi  $t = 1$  đến  $G$  thực hiện
12:    Đánh giá toàn bộ quần thể:  $EXECUTE \rightarrow EVALUATEGRAPH$ 
13:    Nếu  $\max f \geq 0.95$  thì
14:      break ▷ dừng sớm hội tụ
15:    end Nếu
16:    Sinh offspring bằng tournament, crossover, mutation thích nghi
17:    Đánh giá offspring
18:    Chọn sống sót  $(\mu + \lambda)$  theo khả thi-ưu tiên
19:  end Đối với mỗi
20: end Nếu
21: Hậu xử lý đồ thị tốt nhất: sửa tiến trình, lọc virtual node, kiểm tra hợp lệ topology
22: Trả về:  $\mathcal{G}^*$ 

```

luật grammar, đánh giá cá thể, đến bước chọn sống sót và hậu kiểm tra đầu ra.

Tiêu chí tính đúng, điều kiện dừng và độ phức tạp. Với ký hiệu $n = |V|$, $m = |E|$, P là kích thước quần thể, G là số thế hệ và L là độ dài genome, tính đúng của thuật toán được bảo đảm nhờ chuỗi kiểm soát ràng buộc theo ba lớp: dynamic action mask và hard safety cap ở pha tạo phenotype, cờ khả thi $\phi = \mathbf{1}[v \leq 10^{-9}]$ ở pha đánh giá ràng buộc, và hậu kiểm tra hợp lệ topology (liên thông, start/goal, độ dài đường đi tối thiểu, hợp lệ boss-goal, loại virtual node) trước khi trả đồ thị cuối. Quá trình tìm kiếm dừng khi nhánh GA chạy đủ G thế hệ hoặc đạt ngưỡng hội tụ của fitness tốt nhất (≥ 0.95), còn nhánh CVT-emitter dừng khi dùng hết ngân sách $P \cdot G$. Từ các thành phần chi phí thực thi, đánh giá và chọn lọc, độ phức tạp được chặn trên bởi:

$$\mathcal{O}(PG (C_{\text{exec}} + C_{\text{eval}}) + PG \log P) \quad (3.14)$$

cho nhánh GA, và

$$\mathcal{O}(PG (C_{\text{exec}} + C_{\text{eval}})) \quad (3.15)$$

cho nhánh CVT-emitter.

3.3.3 Khối mã hóa điều kiện graph-to-room (Khối III)

Khối III thực hiện ánh xạ từ ngữ cảnh đồ thị nhiệm vụ sang vector điều kiện cho từng phòng đích. Với phòng i , bộ điều kiện hai luồng được viết:

$$c_i^{\text{local}} = f_{\text{local}}([z_N, z_S, z_E, z_W, b_i, p_i]), \quad (3.16)$$

$$H = \text{GNN}(G; X, E, \text{TPE}), \quad c_i^{\text{global}} = H_i, \quad (3.17)$$

$$c_i = \text{Attn}(q = c_i^{\text{local}}, K = H, V = H) + f_{\text{fuse}}([c_i^{\text{local}}, c_i^{\text{global}}]). \quad (3.18)$$

Trong đó $z_{N,S,E,W}$ là latent của các phòng lân cận theo bốn hướng, b_i là ràng buộc biên và p_i là đặc trưng vị trí phòng. Nhánh toàn cục hỗ trợ nhiều biến thể GNN; cấu hình mặc

định ưu tiên GPS cho bài toán đồ thị có tín hiệu toàn cục mạnh [21]. Ngoài ra, các thiên kiến cấu trúc kiểu Graphormer (khoảng cách đường đi ngắn nhất và structural bias theo quan hệ cạnh) cũng là cơ sở lý thuyết để củng cố lớp mã hóa đồ thị trong khối này [25]. Tầng attention đóng vai trò căn chỉnh thông tin toàn cục vào ngữ cảnh cục bộ của từng phòng, từ đó giảm mâu thuẫn giữa ràng buộc biên và vai trò gameplay ở cấp dungeon.

3.3.4 Khối sinh nơ-ron: VQ-VAE (Khối II) và latent diffusion (Khối IV)

Khối II đóng vai trò cầu nối giữa không gian tile rời rạc và không gian latent liên tục, còn Khối IV học quá trình khử nhiễu có điều kiện trên latent theo khung DDPM/DDIM [8, 9]. Ở pha suy luận, mô hình sử dụng classifier-free guidance (CFG), tương ứng với:

$$\tilde{\epsilon}_{\theta}(x_t, t, c) = \epsilon_{\theta}(x_t, t, \emptyset) + s_t(\epsilon_{\theta}(x_t, t, c) - \epsilon_{\theta}(x_t, t, \emptyset)), \quad (3.19)$$

trong đó s_t là hệ số guidance theo lịch runtime và được kẹp trong miền hợp lệ của teacher khi chạy fast sampler [31]. Cơ chế này nhất quán với mã nguồn: nhánh student không được phép dùng mức CFG vượt quá miền đã distill.

Ở pha huấn luyện, diffusion loss được tái trọng số theo Min-SNR để giảm lệch trọng số giữa các timestep:

$$\text{SNR}_t = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}, \quad w_t = \frac{\min(\text{SNR}_t, \gamma)}{\text{SNR}_t + \varepsilon}, \quad \mathcal{L}_{\text{diff}} = \mathbb{E}_{t, \epsilon} [w_t \|\hat{y}_t - y_t\|_2^2], \quad (3.20)$$

với y_t là nhiễu mục tiêu (hoặc vận tốc v khi bật v-prediction) và γ là ngưỡng Min-SNR [32].

3.3.5 Khối hướng dẫn logic và sửa lỗi ký hiệu (Khối V–VI)

Khối V đưa tín hiệu logic vào huấn luyện theo cơ chế *warmup* nhằm tránh gây nhiễu khi mô hình chưa ổn định:

$$\mathcal{L}_{\text{total}} = \alpha_{\text{visual}} \mathcal{L}_{\text{diff}} + \mathbf{1}[e \geq e_{\text{warmup}}] \alpha_{\text{logic}} \mathcal{L}_{\text{logic}}. \quad (3.21)$$

Ở suy luận DDIM-style, guidance logic tác động trực tiếp lên dự đoán nhiễu:

$$\hat{\epsilon}'_{\theta} = \hat{\epsilon}_{\theta} + \sqrt{1 - \bar{\alpha}_t} \nabla_{x_t} \mathcal{L}_{\text{logic}}, \quad (3.22)$$

rồi mới cập nhật lại $\hat{x}_0$. Cách viết này phản ánh đúng thao tác trong Khối IV khi logic guidance được bật.

Sau giải mã, Khối VI xử lý phòng theo chuỗi chuẩn hóa: cleanup, sửa biên và repair có điều kiện theo plan mask. Thành phần WFC-guided inpaint được dùng như cơ chế sửa cục bộ khi phát hiện vùng vi phạm ràng buộc [28]. Do plan mask được suy từ semantics của đồ thị (vai trò phòng, cửa bắt buộc, quan hệ incoming/outgoing), quá trình sửa lỗi không chỉ cải thiện hình học mà còn khôi phục tính đúng đắn của tiến trình gameplay.

Thuật toán này được thiết kế để bảo toàn tính nhất quán giữa tầng nơ-ron và tầng ký hiệu: đầu vào là lưới nơ-ron cùng ngữ cảnh đồ thị phòng, đầu ra là lưới đã chuẩn hóa ngữ nghĩa, thỏa ràng buộc biên-cửa và giữ khả giải cục bộ. Trật tự xử lý được cố định theo chuỗi chuẩn hóa $\rightarrow$ ép biên $\rightarrow$ sửa có phản hồi theo plan mask $\rightarrow$ phủ marker đồ thị $\rightarrow$ kiểm tra hợp lệ, nhờ đó mọi biến đổi đều có thể truy vết và kiểm chứng.

Thuật toán 2 Sửa phòng có phản hồi ký hiệu ở tầng hậu xử lý

Đầu vào: Neural grid x_i^{neural} , graph phòng G_i , start-goal (s_i, g_i)

Đầu ra: Grid cuối x_i^{final} thỏa ràng buộc biên và đường đi

- 1: $x_i^{\text{final}} \leftarrow \text{SANITIZESEMANTICIDS}(x_i^{\text{neural}})$
 - 2: $x_i^{\text{final}} \leftarrow \text{STRIPVOLATILESEMANTICS}(x_i^{\text{final}}, G_i)$
 - 3: $x_i^{\text{final}} \leftarrow \text{ENFORCEBOUNDARYSHELL}(x_i^{\text{final}}, G_i)$
 - 4: $m_i \leftarrow \text{BUILDROOMPLANTRACE}(G_i, s_i, g_i)$
 - 5: **Nếu** bật chế độ sửa hậu kỳ **thì**
 - 6: $x_i^{\text{cand}} \leftarrow \text{REPAIRROOMWITHFEEDBACK}(x_i^{\text{final}}, m_i)$
 - 7: **Nếu** repair thành công **thì**
 - 8: $x_i^{\text{final}} \leftarrow x_i^{\text{cand}}$
 - 9: **end Nếu**
 - 10: **end Nếu**
 - 11: $x_i^{\text{final}} \leftarrow \text{OVERLAYGRAPHMARKERS}(x_i^{\text{final}}, G_i)$
 - 12: $\text{VALIDATEROOMDIMENSIONS}(x_i^{\text{final}})$
 - 13: **Trả về:** x_i^{final}
-

Thuật toán 2 là cầu nối giữa chất lượng thị giác và tính đúng gameplay: nếu không có

lớp này, sai lệch cục bộ về cửa, biên hoặc đường đi sẽ lan sang bước ghép dungeon và làm giảm độ tin cậy của đánh giá toàn cục.

Tiêu chí tính đúng, điều kiện dừng và độ phức tạp. Ở tầng sửa hậu kỳ, tính đúng của đầu ra phòng được bảo đảm khi đồng thời giữ đúng kích thước phòng, thỏa hợp đồng biên-cửa theo ngữ cảnh đồ thị, và không làm suy giảm khả năng cục bộ sau bước repair cùng overlay marker. Thuật toán dừng khi hoàn tất chuỗi thao tác chuẩn hóa hữu hạn; riêng nhánh repair bị chặn trên bởi số vòng thử theo cấu hình runtime. Do đó, độ phức tạp theo phòng là:

$$\mathcal{O}(HW + R C_{\text{rep}}(H, W)), \quad (3.23)$$

với R là số vòng repair thực thi.

3.4 Thuật toán huấn luyện và suy luận trong mã nguồn

3.4.1 Huấn luyện theo giai đoạn

Huấn luyện được tổ chức theo chuỗi VQ-VAE $\rightarrow$ Diffusion $\rightarrow$ Fast Sampler $\rightarrow$ Masked Room, mỗi giai đoạn có checkpoint riêng để phục vụ resume và ablation. Trong giai đoạn diffusion, mỗi batch được xử lý theo các bước: mã hóa room thật sang latent thật, dựng điều kiện graph-to-room đúng schema, tính $\mathcal{L}_{\text{diff}}$ với Min-SNR weighting, sau đó cộng logic loss theo lịch warmup. Các cơ chế an toàn như bỏ qua batch non-finite, gradient clipping và EMA được duy trì trong vòng lặp chính để hạn chế lỗi tối ưu hóa lan truyền theo thời gian huấn luyện.

Ở mức hiện thực, scheduler và checkpoint stage giữ vai trò neo để chuyển pha huấn luyện mà không làm đứt mạch tối ưu hóa. Khi kết hợp với kiểm tra non-finite, gradient clipping và EMA, vòng lặp huấn luyện giữ được cả độ ổn định số học lẫn khả năng truy vết khi resume thí nghiệm.

Để mã giả có tính học thuật và dễ đối chiếu, luận văn theo quy ước trình bày thường

gặp trong các bài báo gốc về diffusion và graph diffusion: tách rõ nhánh train/sampling, nêu tường minh input-output, và đặt tên bước theo hành động thuật toán ở mức trừu tượng (ví dụ: lấy mẫu, gom batch, cập nhật) thay vì dùng trực tiếp tên hàm triển khai trong mã nguồn [8, 9, 26, 22].

Thuật toán 3 Huấn luyện một epoch diffusion với warmup logic

Đầu vào: Dataloader $\mathcal{D}$ trả về $(x, \text{graph_list})$, cấu hình huấn luyện

Đầu ra: Tham số diffusion/condition encoder được cập nhật ổn định

```

1:  $include\_logic \leftarrow (epoch \geq e_{warmup})$ 
2: Với mỗi  $(x, \text{graph\_list}) \in \mathcal{D}$  thực hiện
3:   Mã hóa từng graph:  $c_i \leftarrow \text{ENCODEGRAPHCONDITIONING}(\text{graph\_list}[i])$ 
4:   Gom các vector điều kiện thành  $C_{batch}$ 
5:   Gom các đồ thị thành  $G_{batch}$ 
6:   Tính  $\mathcal{L}_{diff}$  với Min-SNR weighting
7:   Nếu  $include\_logic$  thì
8:     Tính  $\mathcal{L}_{logic}$  trên predicted latent
9:   Ngược lại
10:     $\mathcal{L}_{logic} \leftarrow 0$ 
11:  end Nếu
12:   $\mathcal{L}_{total} \leftarrow \alpha_{visual}\mathcal{L}_{diff} + \alpha_{logic}\mathcal{L}_{logic}$ 
13:  Nếu  $\mathcal{L}_{total}$  hoặc gradient không hữu hạn thì
14:    Bỏ qua batch hiện tại, tăng bộ đếm non-finite, tiếp tục batch kế
15:  continue
16: end Nếu
17: Gradient clipping, OPTIMIZERSTEP, UPDATEEMA
18: end Đối với mỗi
19: Cập nhật scheduler và nhiệt độ LogicNet theo global step

```

Theo nghĩa phương pháp luận, Thuật toán 3 hiện thực trực tiếp nguyên tắc “ổn định trước, ràng buộc sau”: tín hiệu logic được đưa vào có kiểm soát để tránh làm lệch gradient ở giai đoạn mô hình chưa hội tụ đủ.

Trong mã nguồn, hai bước gom batch điều kiện và gom batch đồ thị được hiện thực bằng các helper đóng gói tensor tương ứng trong luồng huấn luyện diffusion.

Tiêu chí tính đúng, điều kiện dừng và độ phức tạp. Ở bước cập nhật tham số, tính đúng được bảo đảm khi vòng lặp huấn luyện duy trì đồng thời ba bất biến vận hành: hàm mất mát và gradient luôn hữu hạn trên từng mini-batch, tensor điều kiện graph-to-room

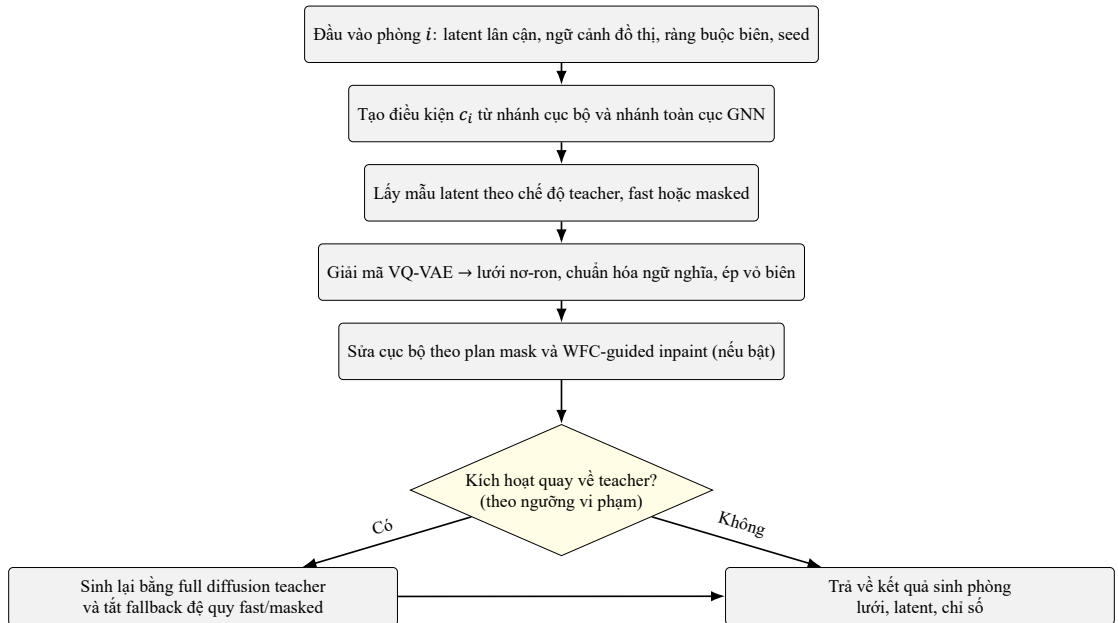
luôn tuân thủ đúng schema dữ liệu đã đặc tả, và thành phần $\mathcal{L}_{\text{logic}}$ chỉ được kích hoạt sau ngưỡng warmup e_{warmup} . Quá trình lặp dừng một cách tự nhiên khi đã duyệt hết dataloader trong một epoch, tương đương $|\mathcal{D}|$ bước cập nhật. Với mô hình chi phí theo batch, độ phức tạp xấp xỉ:

$$\mathcal{O}(|\mathcal{D}| \cdot (C_{\text{enc}} + C_{\text{diff}} + \mathbb{I}_{\text{logic}} C_{\text{logic}})), \quad (3.24)$$

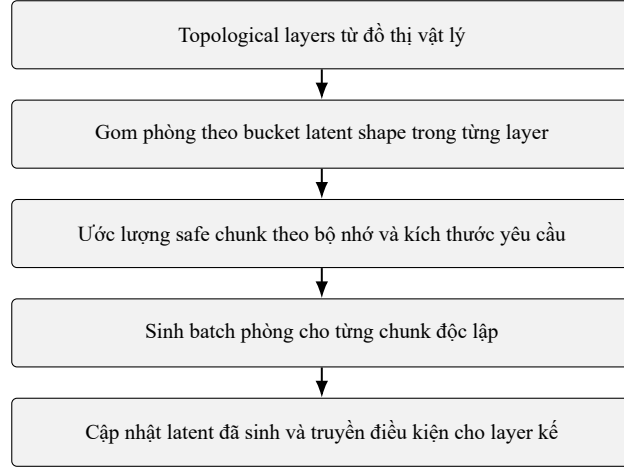
trong đó C_{diff} là hạng chi phối.

3.4.2 Suy luận nhiều phòng và cơ chế fallback

Suy luận dungeon được tổ chức theo ba mức: mức đồ thị, mức lớp phụ thuộc và mức phòng. Ở mức đồ thị, hệ thống chuẩn hóa cấu trúc đồ thị nhiệm vụ và xây dựng ngữ cảnh đồ thị. Ở mức lớp phụ thuộc, các phòng được sắp theo độ sâu topo [29] rồi batch theo latent shape để giữ phụ thuộc logic và tận dụng song song hóa. Ở mức phòng, điều kiện được dựng từ hàng xóm đã sinh, latent được lấy mẫu theo chế độ teacher/fast/masked, sau đó giải mã qua VQ-VAE và áp lớp sửa ký hiệu trước khi trả kết quả.



Hình 3.2: Luồng nội bộ của quy trình sinh phòng, bao gồm sửa lỗi ký hiệu và cơ chế teacher fallback.



Hình 3.3: Luồng lập lịch sinh phòng theo lớp phụ thuộc và bucket latent shape.

Thuật toán 4 Lập lịch sinh phòng theo lớp phụ thuộc

Đầu vào: Đầu vào đã chuẩn bị (G_{phys}, C) , cấu hình sinh phòng

Đầu ra: Tập kết quả phòng $\mathcal{R}$, latent phòng và chẩn đoán batch

- 1: **Nếu** bật chế độ batch phòng độc lập và G_{phys} có hướng **thì**
 - 2: $\mathcal{L} \leftarrow \text{TOPOLOGICALGENERATIONLAYERS}(G_{phys})$
 - 3: **Với mỗi** layer $\ell \in \mathcal{L}$ **thực hiện**
 - 4: $\mathcal{B} \leftarrow \text{BUCKETBYLATENTSHAPE}(\ell)$
 - 5: **Với mỗi** bucket $b \in \mathcal{B}$ **thực hiện**
 - 6: **Nếu** kích thước phòng không chuẩn (16, 11) **thì**
 - 7: Sinh tuần tự từng phòng trong b
 - 8: **Ngược lại**
 - 9: $k \leftarrow \text{ESTIMATESAFEBATCHSIZE}(b, k_{\max})$
 - 10: **Với mỗi** chunk độc lập kích thước k trong b **thực hiện**
 - 11: $\mathcal{R}_{chunk} \leftarrow \text{GENERATEROOMBATCH}(chunk, C)$
 - 12: Cập nhật $\mathcal{R}$ và latent đã sinh
 - 13: **end Đối với mỗi**
 - 14: **end Nếu**
 - 15: **end Đối với mỗi**
 - 16: **end Đối với mỗi**
 - 17: **Ngược lại**
 - 18: Sinh tuần tự theo thứ tự nút ổn định
 - 19: **end Nếu**
 - 20: **Trả về:** $\mathcal{R}$, latent phòng, chẩn đoán batch
-

Bài toán sinh đa phòng được mô hình hóa như bài toán lập lịch có phụ thuộc thứ tự, do đó thuật toán tách xử lý theo topological layer để bảo toàn quan hệ tiền nhiệm-hậu nhiệm. Trong từng lớp, các phòng được gom theo latent shape và chia chunk an toàn theo giới hạn bộ nhớ để tăng mức song song hóa mà không vi phạm ràng buộc logic. Nhánh tuần tự cho kích thước không chuẩn bảo đảm tính tương thích của pipeline.

Tiêu chí tính đúng, điều kiện dừng và độ phức tạp. Đối với lập lịch đa phòng, tính đúng được bảo đảm vì trật tự topological layer bảo toàn phụ thuộc predecessor $\rightarrow$ successor, cơ chế bucket theo latent shape tránh trộn batch khác hình dạng, và các phòng không đúng kích thước chuẩn được chuyển sang nhánh tuần tự để giữ hợp đồng tensor. Thuật toán dừng khi đã duyệt hết tập layer, bucket và chunk trong mỗi bucket. Vì vậy, chi phí phân lập lịch là:

$$\mathcal{O}(n + m) + \mathcal{O}(N), \quad (3.25)$$

và tổng chi phí sinh phòng là:

$$\sum_{b \in \mathcal{B}} \left\lceil \frac{|b|}{k_b} \right\rceil C_{\text{batch}}(k_b), \quad (3.26)$$

với k_b là safe chunk size của bucket b .

Thuật toán điều phối toàn bộ pipeline topology-first từ khâu chuẩn bị đồ thị đến ghép dungeon và đánh giá chất lượng. Cách tổ chức theo các pha có hợp đồng dữ liệu rõ ràng giúp kiểm soát sai lệch xuyên tầng, đồng thời hỗ trợ ablation và kiểm thử thành phần một cách hệ thống. Vì vậy, tính đúng đắn được bảo toàn ở cả cấp phòng lẫn cấp dungeon.

Tiêu chí tính đúng, điều kiện dừng và độ phức tạp. Ở cấp pipeline, tính đúng đòi hỏi đồ thị sinh mới (khi không có đầu vào) phải qua kiểm tra topology trước khi sử dụng, các phòng phải được sinh theo lớp phụ thuộc và ghép trên đồ thị vật lý đã lọc virtual node, đồng thời các chỉ số hậu kỳ phải tách rõ tầng phòng và tầng dungeon. Quy trình dừng sau khi hoàn tất hữu hạn các pha chuẩn bị đồ thị, sinh phòng theo bucket/chunk, ghép

Thuật toán 5 Suy luận dungeon theo cơ chế topology-first

Đầu vào: Đồ thị nhiệm vụ tùy chọn G , siêu tham số suy luận Θ

Đầu ra: Dungeon D , tập phòng $\mathcal{R}$, metrics $\mathcal{M}$

- 1: **Nếu** $G = \emptyset$ và bật chế độ sinh cấu trúc đồ thị **thì**
 - 2: $G \leftarrow \text{EVOLUTIONARYTOPOLOGYGENERATOR}(\Theta_{\text{topology}})$
 - 3: **end Nếu**
 - 4: $G_{\text{phys}} \leftarrow \text{FILTERVIRTUALNODES}(G)$
 - 5: $C \leftarrow \text{PREPAREGRAPHCONTEXT}(G_{\text{phys}})$
 - 6: Chia các phòng theo topological layer và bucket latent shape
 - 7: **Với mỗi** bucket phòng độc lập **thực hiện**
 - 8: Sinh batch phòng bằng $\text{GENERATEROOMBATCH}(C, \Theta)$
 - 9: Cập nhật latent đã sinh để cập điều kiện cho lớp kế
 - 10: **end Đối với mỗi**
 - 11: $D \leftarrow \text{STITCHROOMS}(\mathcal{R}, G_{\text{phys}})$
 - 12: **Nếu** bật đánh giá MAP-Elites **thì**
 - 13: $Q \leftarrow \text{EVALUATEGENERATEDDUNGEON}(D)$
 - 14: **end Nếu**
 - 15: **Trả về:** $(D, \mathcal{R}, \mathcal{M})$
-

dungeon và (tùy chọn) đánh giá QD. Theo đó, độ phức tạp tổng thể có thể viết:

$$\mathcal{O}(C_{\text{topology}} + C_{\text{rooms}} + C_{\text{stitch}} + C_{\text{eval}}), \quad (3.27)$$

trong đó C_{topology} được cho bởi Thuật toán 1.

Luồng suy luận một phòng và luật fallback

Ở mức vi mô, một phòng được xử lý theo chuỗi dựng điều kiện $\rightarrow$ lấy mẫu latent $\rightarrow$ giải mã $\rightarrow$ repair. Luật fallback đóng vai trò cơ chế an toàn để chuyển từ nhánh sampler nhanh về teacher khi chỉ số vi phạm vượt ngưỡng thiết kế.

Cơ chế này cân bằng chi phí suy luận và độ tin cậy cấu trúc bằng cách cho phép các nhánh lấy mẫu nhanh hoạt động dưới giám sát hậu kiểm. Phòng được sinh theo chuỗi điều kiện $\rightarrow$ lấy mẫu latent $\rightarrow$ giải mã $\rightarrow$ sửa cục bộ; sau đó bộ chỉ số vi phạm quyết định có kích hoạt teacher fallback hay không. Thiết kế hậu kiểm này bảo đảm các trường hợp vượt ngưỡng rủi ro được tái sinh bằng cấu hình an toàn hơn.

Thuật toán 6 Sinh một phòng với cơ chế teacher fallback

Đầu vào: Điều kiện phòng $(z_N, z_S, z_E, z_W, b_i, p_i, G_i)$, cấu hình sampler

Đầu ra: Kết quả sinh phòng cho phòng i

- 1: $c_i \leftarrow \text{COMPUTEROOMCONDITION}(z, b_i, p_i, G_i)$
 - 2: **Nếu** mode = masked rời rạc **thì**
 - 3: $z_i, \ell_i \leftarrow \text{MASKEDROOMSAMPLE}(c_i, G_i)$
 - 4: **Ngược lại Nếu** mode = categorical **thì**
 - 5: $z_i, \ell_i \leftarrow \text{CATEGORICALCODEBOOKSAMPLE}(c_i)$
 - 6: **Ngược lại**
 - 7: $z_i \leftarrow \text{DIFFUSIONSAMPLE}(c_i, G_i, \text{CFG}, \text{logic guidance})$
 - 8: $\ell_i \leftarrow \text{VQVAE.DECODE}(z_i)$
 - 9: **end Nếu**
 - 10: $x_i^{\text{neural}} \leftarrow \arg \max \ell_i$
 - 11: $x_i^{\text{final}} \leftarrow \text{REPAIRANDBOUNDARYENFORCE}(x_i^{\text{neural}}, G_i)$
 - 12: **Nếu** SHOULDRETRYROOMWITHTEACHER(x_i^{final} , metrics) **thì**
 - 13: Sinh lại với full diffusion teacher và tắt fallback đệ quy
 - 14: **end Nếu**
 - 15: **Trả về:** result($x_i^{\text{final}}, z_i, \text{metrics}$)
-

Tiêu chí tính đúng, điều kiện dừng và độ phức tạp. Ở cấp một phòng, tính đúng được duy trì vì đầu ra luôn đi qua hậu kiểm tra biên/repair, còn fallback chỉ kích hoạt khi chỉ số vi phạm vượt ngưỡng và khi đó chuyển sang teacher để khôi phục độ tin cậy. Quy trình dừng sau một lượt sinh chính và tối đa một lượt retry teacher (tắt fallback đệ quy), nên số vòng lặp luôn hữu hạn. Do đó, độ phức tạp theo phòng là:

$$\mathcal{O}(C_{\text{sample}} + C_{\text{decode}} + C_{\text{repair}} + \mathbb{I}_{\text{fb}} C_{\text{teacher}}). \quad (3.28)$$

Để hạn chế rủi ro nhiều cấu trúc từ các nhánh suy luận chi phí thấp hơn teacher, quy trình áp dụng luật fallback tường minh. Với một phòng r , cờ fallback được viết gọn như sau:

$$\begin{aligned} n_{\text{island}} \geq 1 \vee n_{\text{door-err}} \geq 1 \\ \mathbb{I}_{\text{fallback}}(r) = \mathbf{1} \left[\vee (n_{\text{repair}} \geq 18 \wedge n_{\text{obs}} > 0) \right. \\ \left. \vee (m = \text{masked} \wedge (r_{\text{overwrite}} > 0.34 \vee n_{\text{repair}} \geq 12)) \right]. \end{aligned} \quad (3.29)$$

Khi điều kiện này đúng, phòng được sinh lại bằng full diffusion teacher. Cơ chế này thể

hiện nguyên tắc vận hành của hệ thống: ưu tiên độ tin cậy đầu ra hơn lợi ích tốc độ cục bộ của từng sampler.

3.4.3 Đánh giá sau sinh và chỉ số vận hành

Sau khi ghép phòng, hệ thống triển khai hai tầng đánh giá bổ sung. Tầng thứ nhất đưa dungeon hợp lệ vào MAP-Elites và trích xuất các chỉ số như linearity, leniency, progression/topology complexity, path length. Tầng thứ hai ước lượng khả giải ở cấp dungeon từ tập phòng đã sinh, tách biệt với loss huấn luyện ở cấp phòng.

Các chỉ số tổng hợp vận hành được ghi trực tiếp vào biến chỉ số ở cuối hàm sinh dungeon, trong đó có:

$$\text{repair_rate} = \frac{1}{N} \sum_{i=1}^N r_i, \quad \text{solvability_score} = \frac{1}{N_{eval}} \sum_{i=1}^{N_{eval}} \text{grid_reach}_i. \quad (3.30)$$

trong đó $r_i = 1$ nếu phòng i có sửa lỗi và $r_i = 0$ nếu không có sửa lỗi. Việc tách chỉ số theo tầng như trên giúp phân biệt rõ lỗi sinh phòng cục bộ, lỗi ghép dungeon và lỗi tiến trình ở cấp đồ thị.

Thuật toán 7 Đánh giá dungeon sau ghép (MAP-Elites + LogicNet)

Đầu vào: Dungeon đã ghép D , tập phòng $\mathcal{R}$, đồ thị vật lý G_{phys}

Đầu ra: Chỉ số QD, khả giải và danh sách phòng lỗi

- 1: **Nếu** bật MAP-Elites và thành phần MAP-Elites khả dụng **thì**
 - 2: $s \leftarrow \text{VALIDATEDUNGEON}(D)$
 - 3: **Nếu** s thỏa điều kiện giải được **thì**
 - 4: $\text{MAPELITESADDDUNGEON}(D, s, G_{phys})$
 - 5: Trích xuất các chỉ số QD từ s
 - 6: **end Nếu**
 - 7: **end Nếu**
 - 8: **Nếu** LogicNet khả dụng **thì**
 - 9: **Với mỗi** phòng $r_i \in \mathcal{R}$ có latent **thực hiện**
 - 10: $(\ell_i, info_i) \leftarrow \text{LOGICNET}(z_i)$
 - 11: Cộng dồn grid_reach_i và ghi phòng lỗi nếu dưới ngưỡng
 - 12: **end Đối với mỗi**
 - 13: **end Nếu**
 - 14: **Trả về:** các chỉ số đánh giá và danh sách phòng lỗi
-

Đánh giá hậu sinh được phân tầng để tách bạch chất lượng cấu trúc toàn dungeon và khả giải ở cấp phòng. MAP-Elites cung cấp góc nhìn quality-diversity ở mức archive, trong khi LogicNet lượng hóa tín hiệu khả giải từ latent từng phòng. Cách tách điều kiện theo thành phần khả dụng giúp quy trình vẫn ổn định trong các thiết lập ablation mà không làm mất tính so sánh.

Tiêu chí tính đúng, điều kiện dừng và độ phức tạp. Ở tầng đánh giá hậu sinh, tính đúng yêu cầu dungeon chỉ được thêm vào MAP-Elites sau khi qua kiểm tra hợp lệ tổng thể và có trạng thái giải được; đồng thời ở tầng phòng, điểm LogicNet được tích lũy theo từng phòng có latent để phát hiện phòng lỗi một cách nhất quán. Quy trình dừng khi kết thúc hai vòng lặp hữu hạn gồm kiểm tra dungeon tổng thể và duyệt danh sách phòng hợp lệ để chấm logic. Vì vậy, độ phức tạp là:

$$\mathcal{O}(C_{\text{validate}}(D) + N_z C_{\text{logicnet}} + C_{\text{archive}}). \quad (3.31)$$

3.4.4 Giả định vận hành, chế độ lỗi và biện pháp giảm thiểu

Giả định vận hành. Mô tả phương pháp trong chương này dựa trên ba giả định vận hành cơ bản: (i) đồ thị vật lý sau khi lọc nút ảo vẫn liên thông và có cặp start-goal hợp lệ, (ii) phòng đầu vào/đầu ra tuân thủ hợp đồng kích thước và từ vựng tile, và (iii) kênh điều kiện biên-cửa được dựng nhất quán với quan hệ hàng xóm. Viết gọn:

$$\text{Connected}(G_{\text{phys}}) = 1, \quad s, g \in V_{G_{\text{phys}}}, s \neq g, \quad (3.32)$$

$$x_i \in \{0, \dots, 43\}^{16 \times 11}, \quad r_d \leq h_d \quad (d \in \{N, S, E, W\}). \quad (3.33)$$

Các giả định này không thay thế đánh giá thực nghiệm, nhưng giúp xác định rõ miền hiệu lực của các thuật toán ở Mục 3.4.

Chế độ lỗi và giảm thiểu. Bảng 3.3 tổng hợp các lỗi vận hành thường gặp và cơ chế giảm thiểu đã cài trong pipeline.

Bảng 3.3: Các chế độ lỗi chính và cơ chế giảm thiểu trong đường chạy Chương 3

Nhóm lỗi	Biểu hiện quan sát được	Cơ chế giảm thiểu trong pipeline
Sai schema tensor	Lỗi shape ở conditioning hoặc batch diffusion.	Fail-fast tại điểm vào khối B_3/B_4 , kiểm tra kiểu-kích thước trước khi lấy mẫu.
Topology không khả thi	Đồ thị vi phạm ràng buộc tiến trình hoặc không đạt điều kiện hậu kiểm.	Chọn nghiệm khả thi-ưu tiên, phạt vi phạm mềm, hậu kiểm tra trước khi ghép phòng.
Nhiều cấu trúc từ sampler nhanh	Xuất hiện island, lỗi cửa, hoặc overwrite cao ở masked mode.	Luật teacher fallback theo ngưỡng vi phạm, đồng thời chặn fallback đệ quy.
Lệch biên-cửa sau giải mã	Cửa bắt buộc mất hoặc biên ngoài không kín.	Enforce boundary shell, plan mask repair, WFC-guided inpaint.
Suy giảm thành phần phụ trợ	LogicNet hoặc MAP-Elites tạm không khả dụng ở một số cấu hình ablation.	Chạy suy giảm có kiểm soát: giữ đường chạy chính, tách rõ chỉ số bị vô hiệu hóa.

3.5 Cấu hình triển khai và khả năng tái lập

Cấu hình chuẩn được quản lý tập trung trong bộ tham số thí nghiệm, trong đó các tham số lỗi như kích thước phòng, số lớp tile, latent dimension, CFG scale, số bước diffusion và cờ bật/tắt repair đều được khai báo tường minh. Cơ chế seed được đồng bộ cho các thành phần sinh ngẫu nhiên chính của hệ thống; trong thiết lập phân tán, mỗi worker dùng seed lệch theo chỉ số tiến trình để tách luồng ngẫu nhiên nhưng vẫn bảo toàn khả năng truy vết. Hệ checkpoint được tổ chức theo stage, lưu kèm metadata kiến trúc và trạng thái huấn luyện, đồng thời dùng cơ chế lưu an toàn để giảm nguy cơ hỏng

checkpoint trong các phiên chạy dài.

Về tiêu chí dừng, tầng cấu trúc đồ thị nhiệm vụ có thể dừng sớm khi fitness hội tụ; tầng diffusion dừng theo số epoch cấu hình và chọn checkpoint theo validation metric; tầng suy luận dừng theo số phòng cùng giới hạn số vòng repair. Cách tổ chức này tạo ra đường chạy nhất quán giữa huấn luyện và suy luận, đồng thời giữ chi phí tính toán trong ngưỡng phù hợp với quy mô dữ liệu hiện có.

3.6 Ánh xạ nội dung phương pháp vào luồng hiện thực

Để bảo đảm cấu trúc Chương 3 nhất quán với trật tự vận hành của hệ thống, các mục được ánh xạ theo bốn cụm chức năng: Mục 3.2.1–3.2.3 làm rõ hai lớp đồ thị cùng ma trận hợp đồng khối, Mục 3.2.4–3.2.7 chốt luồng dữ liệu, tiền xử lý và bất biến vận hành; Mục 3.3 nêu nguyên tắc chọn thiết kế rồi đặc tả thuật toán từng khối; Mục 3.4 mô tả quy trình huấn luyện/suy luận kèm giả định vận hành và chế độ lỗi; và Mục 3.5 chốt lớp cấu hình – tái lập. Cách phân tách này giúp người đọc truy vết từ thiết kế khái niệm đến hành vi thực thi mà không phải chuyển ngữ cảnh giữa các tầng.

Ở mức kiểm chứng, cùng một cấu trúc trên cũng tạo điều kiện cho phân tích ablation theo khối: topology prior (Khối I), conditioning (Khối III), diffusion (Khối IV), logic guidance (Khối V), symbolic repair (Khối VI) và quality-diversity evaluation (Khối VII). Do đó, phần phương pháp giữ được cả tính diễn giải học thuật và tính khả kiểm trong mã nguồn.

3.7 Liên kết với thiết kế thực nghiệm Chương 4

Thiết kế phương pháp trong chương này tạo ra trực tiếp các trục kiểm chứng của Chương 4. Do từng khối có giao diện rõ ràng và cờ bật/tắt độc lập, thí nghiệm ablation có thể ước lượng minh bạch đóng góp biên của logic guidance, topology prior, symbolic repair và teacher fallback. Việc đặt các nhánh teacher, fast và masked trong cùng một

quy trình với cơ chế kẹp guidance theo miền distillation giúp so sánh matched-budget công bằng hơn ở cấp độ vận hành. Đồng thời, chuẩn hóa seed, checkpoint và logging cho phép triển khai đánh giá OOD cùng suy luận thống kê theo cặp (bootstrap và FDR) với độ lặp lại cao [12, 13].

3.8 Tóm tắt chương

Chương 3 đã đặc tả đầy đủ quy trình topology-first neural-symbolic ở mức triển khai: từ biểu diễn dữ liệu, cơ chế điều kiện hóa graph-to-room, công thức huấn luyện/suy luận của diffusion, đến các lớp repair và fallback nhằm bảo toàn tính hợp lệ gameplay. Điểm cốt lõi của phương pháp là phân tách trách nhiệm theo tầng nhưng liên kết bằng hợp đồng dữ liệu thống nhất; nhờ đó hệ thống vừa kiểm soát được tiến trình toàn cục, vừa duy trì chất lượng hình học ở mức cục bộ. Trên nền đặc tả này, Chương 4 có thể đánh giá định lượng đóng góp của từng thành phần trong các thiết lập matched-budget và OOD.

CHƯƠNG 4

Kết quả thực nghiệm và thảo luận

Chương này trình bày kế hoạch thực nghiệm và khung phân tích kết quả theo đúng các trục kiểm chứng đã thiết kế ở Chương 1 và Chương 3. Nội dung được tổ chức theo mạch: mục tiêu kiểm chứng → thiết kế giao thức → kết quả định lượng → thảo luận cơ chế và giới hạn.

4.1 Mục tiêu kiểm chứng và câu hỏi thực nghiệm

Phần này đặt lại các câu hỏi nghiên cứu ở dạng kiểm chứng thực nghiệm để làm rõ tiêu chí đánh giá thành công của phương pháp:

- (RQ1) Cấu hình *topology-first neural-symbolic* có cải thiện đồng thời tính hợp lệ gameplay, khả năng giải được và chất lượng phòng so với các cấu hình đối chứng hay không?
- (RQ2) Đóng góp biên của từng khối (topology prior, conditioning, diffusion, logic guidance, symbolic repair, fallback) có ý nghĩa thống kê hay không?
- (RQ3) Khi bị ràng buộc cùng ngân sách đánh giá, phương pháp đề xuất có giữ lợi thế so với các baseline sinh đồ thị nhiệm vụ hay không?
- (RQ4) Hiệu năng có duy trì ổn định trên các kịch bản OOD-small và OOD-large theo quy mô số phòng hay không?

4.2 Thiết kế thực nghiệm

4.2.1 Thiết lập tái lập và môi trường thực nghiệm

Mục này nêu chi tiết phần cứng/phần mềm, phiên bản thư viện, seed, chiến lược checkpoint và logging. Các cấu hình được cố định để bảo đảm mỗi thí nghiệm có thể chạy lại và đối chiếu trực tiếp.

4.2.2 Dữ liệu và kịch bản đánh giá

Mục này mô tả tập huấn luyện/kiểm định và cách dựng các kịch bản đánh giá ID, OOD-small, OOD-large. Phần này cũng chốt nguyên tắc không rò rỉ dữ liệu giữa các split ở cấp mission graph.

4.2.3 Baseline và cấu hình so sánh

Mục này liệt kê các baseline theo từng tầng (topology, room generation, repair), cùng các cấu hình ablation bật/tắt khối trong pipeline đề xuất để so sánh công bằng.

4.2.4 Chỉ số đánh giá và suy luận thống kê

Mục này đặc tả bộ chỉ số chính: hợp lệ, giải được, chất lượng, đa dạng, repair rate, fallback rate và chi phí suy luận. Suy luận thống kê dùng paired bootstrap, kiểm định hoán vị dấu theo cặp và hiệu chỉnh Benjamini-Hochberg FDR.

4.2.5 Giao thức thí nghiệm

Mục này mô tả ba nhánh thí nghiệm lỗi:

- (a) Fixed-seed ablation theo khối chức năng.
- (b) Matched-budget benchmark giữa các phương pháp sinh đồ thị nhiệm vụ.
- (c) OOD scaling theo quy mô dungeon và số phòng.

4.3 Kết quả định lượng

4.3.1 Kết quả tổng thể so với baseline

Trình bày bảng kết quả chính và so sánh định lượng giữa phương pháp đề xuất với các baseline theo từng nhóm chỉ số.

4.3.2 Kết quả ablation theo khối pipeline

Phân tích mức suy giảm khi tắt từng thành phần để lượng hóa đóng góp biên và chỉ ra các khối ảnh hưởng mạnh nhất tới độ tin cậy gameplay.

4.3.3 Kết quả matched-budget ở tầng đồ thị nhiệm vụ

So sánh công bằng theo cùng ngân sách đánh giá để làm rõ ưu/nhược của GA, CVT/CMA-emitter và cấu hình đề xuất trong chất lượng-đa dạng.

4.3.4 Kết quả OOD theo quy mô dungeon

Báo cáo mức biến động chỉ số khi chuyển từ ID sang OOD-small/OOD-large, đồng thời chỉ ra thành phần nào giúp hệ giữ ổn định tốt hơn.

4.3.5 Độ tin cậy vận hành và chi phí

Phân tích repair rate, fallback rate, lỗi hậu kiểm và thời gian suy luận để đánh giá đánh đổi giữa độ tin cậy và chi phí tính toán.

4.4 Thảo luận

4.4.1 Giải thích cơ chế cải thiện

Liên hệ trực tiếp kết quả thực nghiệm với cơ chế phương pháp của Chương 3 để giải thích vì sao pipeline đạt cải thiện hoặc suy giảm trong từng thiết lập.

4.4.2 Đánh đổi chất lượng, đa dạng và chi phí

Thảo luận các điểm đánh đổi khi thay đổi cấu hình sampler, mức logic guidance và chính sách fallback, từ đó đề xuất cấu hình khuyến nghị cho từng mục tiêu sử dụng.

4.4.3 Mối đe dọa tính hiệu lực và giới hạn

Nêu các nguồn sai lệch tiềm tàng (độ nhỏ dữ liệu, phụ thuộc seed, giới hạn benchmark, phụ thuộc bộ ràng buộc) và phạm vi mà kết luận thực nghiệm có thể khái quát.

4.5 Tóm tắt chương

Chương 4 tổng hợp kết quả kiểm chứng theo bốn trục: hiệu năng tổng thể, đóng góp thành phần, công bằng ngân sách và khả năng khái quát OOD. Các kết quả này là cơ sở trực tiếp cho phân kết luận, giới hạn và định hướng mở rộng ở Chương 5.

CHƯƠNG 5

Kết luận và hướng phát triển

Tổng kết đóng góp và đề xuất mở rộng.

PHỤ LỤC

A Khung ký hiệu và mô hình chi phí

A.1 Ký hiệu sử dụng trong phụ lục

Ngoài các ký hiệu đã nêu ở Chương 3, phụ lục dùng thêm các đại lượng sau để tường minh hóa phép đếm chi phí:

$$C_{\text{enc}}, C_{\text{diff}}, C_{\text{logic}}, C_{\text{batch}}, C_{\text{teacher}}, C_{\text{archive}}, C_{\text{validate}}. \quad (1)$$

Trong đó mỗi đại lượng biểu thị chi phí thời gian của một khối chức năng tương ứng trong một lần gọi ở mức trừu tượng thuật toán.

A.2 Mô hình chi phí chuẩn hóa

Phân tích trong phụ lục dùng mô hình chi phí chuẩn hóa theo số phép toán sơ cấp và số lần gọi khối xử lý. Mô hình này nhằm so sánh thứ bậc tăng trưởng giữa các thuật toán, không thay cho đo thời gian chạy trên phần cứng cụ thể.

B Chứng minh độ phức tạp

B.1 Bổ đề phân lớp topo

Bổ đề B.1. Phân lớp topo cho đồ thị có hướng bằng phương pháp dựa trên bậc vào có chi phí

$$T_{\text{topo}}(n, m) = \Theta(n + m). \quad (2)$$

Chứng minh. Mỗi nút được đưa vào và lấy ra khỏi hàng đợi đúng một lần, nên tổng chi phí xử lý nút là $\Theta(n)$. Mỗi cạnh được duyệt đúng một lần khi giảm bậc vào của đỉnh đích, nên tổng chi phí xử lý cạnh là $\Theta(m)$. Do đó tổng chi phí là $\Theta(n + m)$ [29]. ■

B.2 Mệnh đề cho sinh cấu trúc đồ thị tiến hóa

Mệnh đề B.2 (nhánh GA). Với kích thước quần thể P và số thế hệ G , chi phí cận trên của Thuật toán 1 ở nhánh GA là

$$T_{GA} = \mathcal{O}(PG (C_{\text{exec}} + C_{\text{eval}}) + PG \log P). \quad (3)$$

Chứng minh. Ở mỗi thế hệ, phân đánh giá cá thể có chi phí tuyến tính theo số cá thể, tức $\mathcal{O}(P(C_{\text{exec}} + C_{\text{eval}}))$. Phần chọn lọc và sắp thứ tự khả thi-ưu tiên có cận trên $\mathcal{O}(P \log P)$. Nhân với G thế hệ thu được biểu thức cần chứng minh. ■

Mệnh đề B.3 (nhánh CVT-emitter). Với ngân sách đánh giá $B = P \cdot G$, chi phí cận trên là

$$T_{\text{CVT}} = \mathcal{O}(PG (C_{\text{exec}} + C_{\text{eval}})). \quad (4)$$

Chứng minh. Mỗi lượt đánh giá gọi hữu hạn các bước thực thi và chấm điểm đồ thị; tổng số lượt bị chặn trên bởi ngân sách $B = P \cdot G$. Vì vậy chi phí tổng thể tỉ lệ tuyến tính theo PG . ■

B.3 Mệnh đề cho lập lịch nhiều phòng

Mệnh đề B.4. Chi phí lập lịch của Thuật toán 4 được chặn trên bởi

$$T_{\text{sched}} = \mathcal{O}(n + m) + \mathcal{O}(N) + \sum_{b \in \mathcal{B}} \left\lceil \frac{|b|}{k_b} \right\rceil C_{\text{batch}}(k_b). \quad (5)$$

Chứng minh. Thành phần thứ nhất đến từ phân lớp topo [29]; thành phần thứ hai đến từ duyệt và gom N phòng vào bucket theo latent shape; thành phần thứ ba đến từ số chunk thực thi trong từng bucket. Tổng ba hạng cho cận trên toàn bộ lịch sinh. ■

B.4 Mệnh đề cho huấn luyện một epoch diffusion

Mệnh đề B.5. Với $|\mathcal{D}|$ batch trong một epoch, chi phí của Thuật toán 3 có dạng

$$T_{\text{epoch}} = \sum_{j=1}^{|\mathcal{D}|} \left(C_{\text{enc}}^{(j)} + C_{\text{diff}}^{(j)} + \mathbb{I}_{\text{logic}}^{(j)} C_{\text{logic}}^{(j)} \right) + \mathcal{O}(|\mathcal{D}|). \quad (6)$$

Nếu chặn trên theo batch tệ nhất, ta thu được

$$T_{\text{epoch}} = \mathcal{O}(|\mathcal{D}| \cdot (C_{\text{enc}} + C_{\text{diff}} + \mathbb{I}_{\text{logic}} C_{\text{logic}})). \quad (7)$$

Chứng minh. Mỗi batch thực hiện đúng một vòng mã hóa điều kiện, một vòng tính loss diffusion, có thể cộng thêm logic loss, rồi bước cập nhật tham số. Cộng tuyến tính theo số batch cho ra biểu thức trên. ■

B.5 Mệnh đề cho suy luận một phòng với teacher fallback

Mệnh đề B.6. Do cơ chế fallback bị chặn tối đa một lần thử lại, chi phí mỗi phòng có cận trên hữu hạn:

$$T_{\text{room}} \leq C_{\text{cond}} + C_{\text{sample}} + C_{\text{decode}} + C_{\text{repair}} + \mathbb{I}_{\text{fb}} (C_{\text{teacher}} + C_{\text{decode}} + C_{\text{repair}}). \quad (8)$$

Chứng minh. Thuật toán có đúng một lượt sinh chính; khi vi phạm ngưỡng thì kích hoạt thêm một lượt teacher và tắt đệ quy fallback. Số lượt bổ sung vì thế bị chặn trên bởi 1, suy ra cận trên hữu hạn như công thức. ■

B.6 Mệnh đề cho đánh giá hậu kỳ sau ghép dungeon

Mệnh đề B.7. Chi phí của Thuật toán 7 là

$$T_{\text{post}} = \mathcal{O}(C_{\text{validate}}(D) + N_z C_{\text{logicnet}} + C_{\text{archive}}). \quad (9)$$

Chứng minh. Khối kiểm tra tổng thể gọi một lần trên dungeon đã ghép; khối chấm logic

duyet tuyến tính theo N_z phòng có latent; phần cập nhật archive được gom vào C_{archive} . Tổng ba hạng cho cận trên yêu cầu. ■

C Phép tính minh họa cho độ phức tạp thực toán

C.1 Nguyên tắc tính minh họa

Các phép tính dưới đây dùng đơn vị chi phí chuẩn hóa để minh họa cách áp dụng công thức; chúng không thay thế benchmark thời gian chạy trên phần cứng thực.

C.2 Ví dụ 1: chi phí sửa một phòng

Với kích thước phòng chuẩn $(H, W) = (16, 11)$, ta có

$$HW = 16 \times 11 = 176. \quad (10)$$

Giả sử số vòng sửa tối đa $R_{\text{max}} = 5$ và mỗi vòng sửa tuyến tính theo số ô, tức $C_{\text{rep}}(H, W) = HW$, khi đó:

$$T_{\text{repair-room}} \leq HW + R_{\text{max}} C_{\text{rep}}(H, W) = 176 + 5 \times 176 = 1056. \quad (11)$$

Nghĩa là cận trên minh họa tương ứng khoảng 1056 lượt xử lý ô lưới cho một phòng.

C.3 Ví dụ 2: chi phí lập lịch nhiều phòng

Giả sử dungeon có $N = 11$ phòng chuẩn, kích thước chunk an toàn $k = 8$, toàn bộ phòng thuộc một bucket chính. Khi đó số lần gọi sinh batch là:

$$\left\lceil \frac{N}{k} \right\rceil = \left\lceil \frac{11}{8} \right\rceil = 2. \quad (12)$$

Nếu đồ thị vật lý có $(n, m) = (11, 15)$, phân lập lịch topo-bucket có chi phí chuẩn hóa xấp xỉ:

$$(n + m) + N = 11 + 15 + 11 = 37, \quad (13)$$

và tổng chi phí xấp xỉ:

$$T_{\text{sched}} \approx 37 + 2 C_{\text{batch}}(8). \quad (14)$$

C.4 Ví dụ 3: ngân sách đánh giá ở tầng đồ thị nhiệm vụ

Chọn cấu hình minh họa $P = 64$, $G = 80$. Tổng số lượt đánh giá cá thể là:

$$P \cdot G = 64 \times 80 = 5120. \quad (15)$$

Với thành phần sắp thứ tự có bậc $PG \log_2 P$, ta có:

$$PG \log_2 P = 5120 \times \log_2(64) = 5120 \times 6 = 30720. \quad (16)$$

Kết quả này minh họa vì sao hạng $PG \log P$ có thể đáng kể khi P tăng.

C.5 Ví dụ 4: kỳ vọng chi phí khi có fallback

Đặt

$$T_0 = C_{\text{cond}} + C_{\text{sample}} + C_{\text{decode}} + C_{\text{repair}}, \quad \Delta = C_{\text{teacher}} + C_{\text{decode}} + C_{\text{repair}}. \quad (17)$$

Nếu xác suất kích hoạt fallback là p_{fb} , kỳ vọng chi phí một phòng là

$$\mathbb{E}[T_{\text{room}}] = T_0 + p_{\text{fb}} \Delta. \quad (18)$$

Ví dụ minh họa với $T_0 = 14$, $\Delta = 12$, $p_{\text{fb}} = 0.25$:

$$\mathbb{E}[T_{\text{room}}] = 14 + 0.25 \times 12 = 17. \quad (19)$$

Điều này cho thấy teacher fallback làm tăng chi phí kỳ vọng theo tuyến tính với xác suất kích hoạt, đổi lại cải thiện biên an toàn chất lượng đầu ra.

Tài liệu tham khảo

- [1] J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne, “Search-based procedural content generation: A taxonomy and survey,” *IEEE Trans. Comput. Intell. AI Games*, vol. 3, no. 3, pp. 172–186, 2011.
- [2] J. Dormans and S. Bakkes, “Generating missions and spaces for adaptable play experiences,” *IEEE Trans. Comput. Intell. AI Games*, vol. 3, no. 3, pp. 216–228, 2011.
- [3] A. Summerville, S. Snodgrass, M. Mateas, and S. Ontanon, “The video game level corpus: A new resource for research on procedural content generation,” in *Proc. AAAI Conf. Artif. Intell. Interactive Digit. Entertainment*, 2016.
- [4] L. T. Pereira, P. V. de Souza Prado, R. M. Lopes, and C. F. M. Toledo, “Procedural generation of dungeons’ maps and locked-door missions through an evolutionary algorithm validated with players,” *Expert Syst. Appl.*, vol. 180, p. 115009, 2021.
- [5] R. Hu, Z. Huang, Y. Tang, O. V. Kaick, H. Zhang, and H. Huang, “Graph2Plan: Learning floorplan generation from layout graphs,” *ACM Trans. Graph.*, vol. 39, no. 4, 2020.
- [6] M. Shabani, H. Aghabozorgi, H. R. Tizhoosh, and N. Miresmaeili, “HouseDiffusion: Vector floorplan generation via a diffusion model with discrete and continuous denoising,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, pp. 5466–5475, 2023.
- [7] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, “Neural discrete representation learning,” in *Adv. Neural Inf. Process. Syst.*, vol. 30, pp. 6306–6315, 2017.

- [8] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 6840–6851, 2020.
- [9] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” in *Proc. Int. Conf. Learn. Represent.*, 2021.
- [10] A. Khalifa, P. Bontrager, S. Earle, and J. Togelius, “PCGRL: Procedural content generation via reinforcement learning,” *Proc. AAAI Conf. Artif. Intell. Interactive Digit. Entertainment*, vol. 16, no. 1, pp. 95–101, 2020.
- [11] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, pp. 10674–10685, 2022.
- [12] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1994.
- [13] Y. Benjamini and Y. Hochberg, “Controlling the false discovery rate: A practical and powerful approach to multiple testing,” *J. R. Stat. Soc. Series B (Methodological)*, vol. 57, no. 1, pp. 289–300, 1995.
- [14] N. Shaker, J. Togelius, and M. J. Nelson, *Procedural Content Generation in Games: A Textbook and an Overview of Current Research*. Springer, 2016.
- [15] A. Summerville, S. Snodgrass, M. Mateas, S. Ontanon, J. Togelius, N. Wardrip-Fruin, and J. Whitehead, “Procedural content generation via machine learning (PCGML),” *IEEE Trans. Games*, vol. 10, no. 3, pp. 257–270, 2018.
- [16] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [17] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.

- [18] J. Austin, D. D. Johnson, J. Ho, D. Tarlow, and R. van den Berg, “Structured denoising diffusion models in discrete state-spaces,” *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 17981–17993, 2021.
- [19] N. Inoue, K. Kikuchi, E. Simo-Serra, M. Otani, and K. Yamaguchi, “LayoutDM: Discrete diffusion model for controllable layout generation,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, pp. 10167–10176, 2023.
- [20] T. Park, M.-Y. Liu, T.-C. Wang, and J.-Y. Zhu, “Semantic image synthesis with spatially-adaptive normalization,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, pp. 2332–2341, 2019.
- [21] L. Rampasek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini, “Recipe for a general, powerful, scalable graph transformer,” in *Adv. Neural Inf. Process. Syst.*, vol. 35, pp. 14501–14515, 2022.
- [22] J.-B. Mouret and J. Clune, “Illuminating search spaces by mapping elites,” *arXiv preprint arXiv:1504.04909*, 2015.
- [23] V. Vassiliades, K. Chatzilygeroudis, and J.-B. Mouret, “Using centroidal voronoi tessellations to scale up the multidimensional archive of phenotypic elites algorithm,” *IEEE Trans. Evol. Comput.*, vol. 22, no. 4, pp. 623–630, 2018.
- [24] M. C. Fontaine, J. Togelius, S. Nikolaidis, and A. K. Hoover, “Covariance matrix adaptation for the rapid illumination of behavior space,” in *Proc. Genet. Evol. Comput. Conf.*, pp. 94–102, 2020.
- [25] C. Ying, T. Cai, S. Luo, S. Zheng, G. Ke, D. He, Y. Shen, and T.-Y. Liu, “Do transformers really perform bad for graph representation?,” in *Adv. Neural Inf. Process. Syst.*, vol. 34, 2021.

- [26] C. Vignac, I. Krawczuk, A. Siraudin, B. Wang, V. Cevher, and P. Frossard, “Diggess: Discrete denoising diffusion for graph generation,” in *Proc. Int. Conf. Learn. Represent.*, 2023.
- [27] G. Rodriguez-Torrado, M. Khalifa, J. Togelius, D. Perez-Liebana, and R. Ruiz, “Bootstrapping conditional GANs for video game level generation,” in *Proc. IEEE Conf. Games (CoG)*, pp. 41–48, 2020.
- [28] I. Karth and A. M. Smith, “WaveFunctionCollapse is constraint solving in the wild,” in *Proc. Int. Conf. Found. Digit. Games*, 2021.
- [29] A. B. Kahn, “Topological sorting of large networks,” *Commun. ACM*, vol. 5, no. 11, pp. 558–562, 1962.
- [30] K. Deb, “An efficient constraint handling method for genetic algorithms,” *Comput. Methods Appl. Mech. Eng.*, vol. 186, no. 2–4, pp. 311–338, 2000.
- [31] J. Ho and T. Salimans, “Classifier-free diffusion guidance,” *arXiv preprint arXiv:2207.12598*, 2022.
- [32] T. Hang, S. Gu, C. Li, J. Bao, D. Chen, H. Hu, X. Geng, and B. Guo, “Efficient diffusion training via Min-SNR weighting strategy,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023.